

COMP 424 - Artificial Intelligence

Fall 2026

Lecture 2: September 2nd, 2026

Uninformed Search

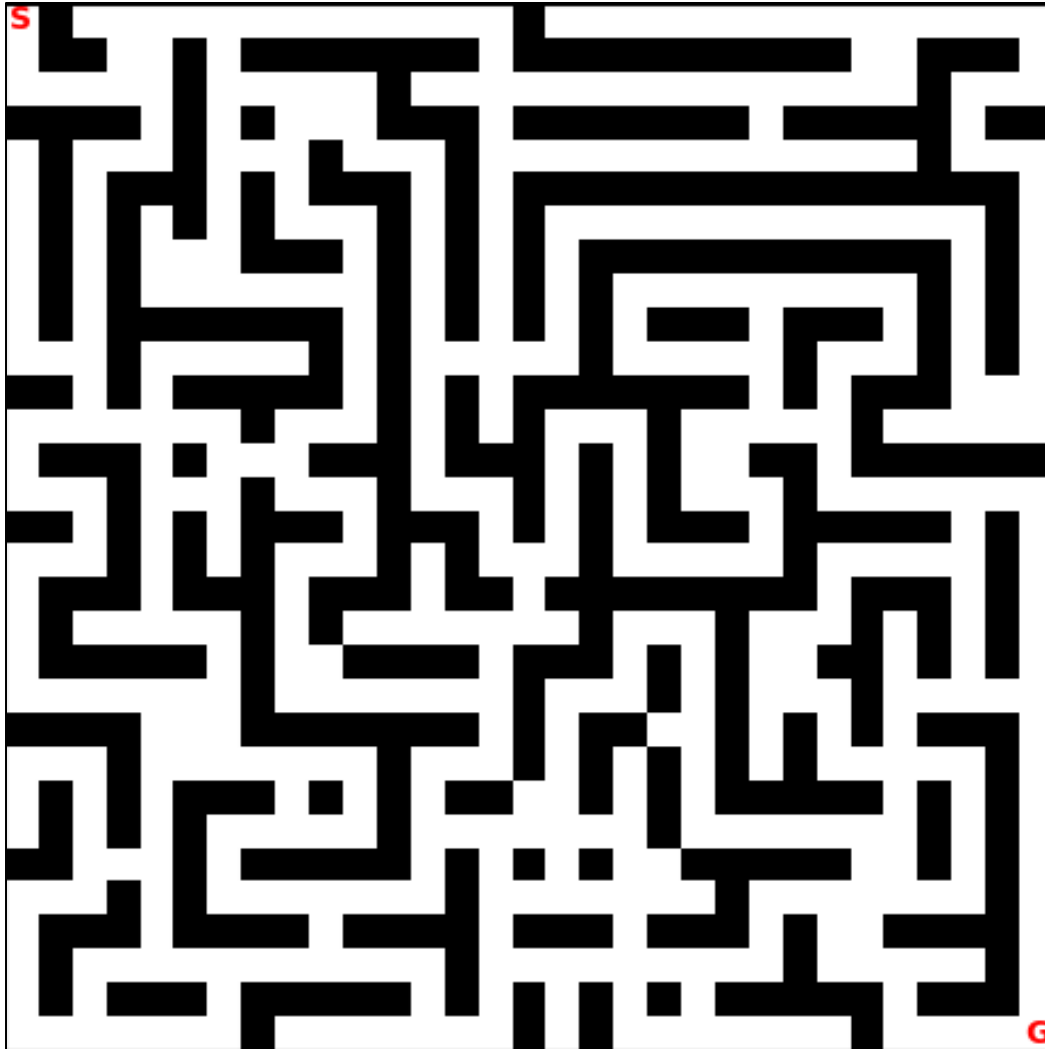
Su Lin Blodgett (sulin.blodgett@mcgill.ca)

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Announcements

- **Ed** is up and running. You all should have been added.
 - Please contact us if you're registered for the course, but you haven't received an email from Ed.
- Office hours are now scheduled:
 - Su Lin: Mondays, 2:00pm - 3:30pm in McConnell 312
 - Jackie: Wednesdays, 9:30am – 11am in McConnell 108N
- **Starting next week (Sept 9), we'll be in MCMED 522!**

The Goal Today



Search in AI

Search is *at the heart* of all AI systems!

Thinking Humanly	Thinking Rationally
Acting Humanly	Acting Rationally

do the right thing

OR

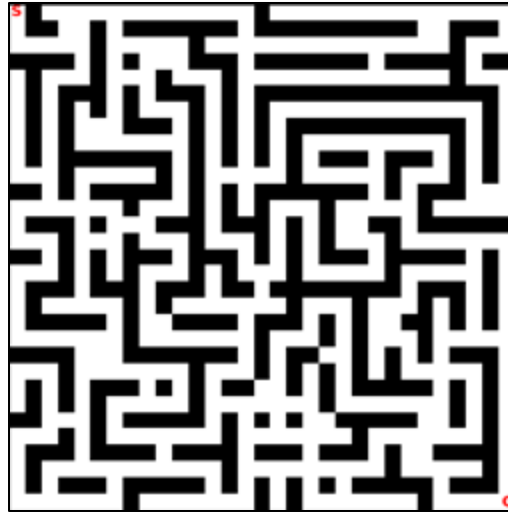
*search for what can be done
and then choose the right thing*

Search in AI

Typical setup of search in AI:

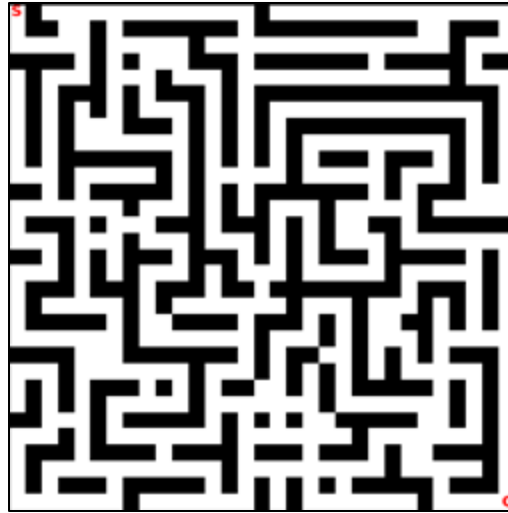
- **Knowledge:** Formal representation of the problem
 - **State:** where are we? **S:** state space
 $s_0 \in S$: initial state
 - **Actions:** what can we do in a state?
(also called Operators) **A:** set of actions
 - **Transitions:** how will the state change if we do an action? **T:** $(s, a) \rightarrow s'$
 - **Goals:** how will we know we're done? **G** $\subset$ **S:** set of goal states
 - **Costs/rewards:** which 'path' to the goal is better? **C:** cost of paths
- **Search:** Algorithmic component that finds a solution

Example: Maze



- States?
- Actions?
- Transitions?
- Goals?
- Path cost?

Example: Maze



- **States?** The maze and our location in the maze
- **Actions?** Up, down, right, and left (where there isn't a wall)
- **Transitions?** Our location changes based on the action
- **Goals?** Reaching the 'End'
- **Path cost?** Length of the path

Example: Eight-Puzzle

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

- States?
- Actions?
- Transitions?
- Goals?
- Path cost?

Example: Eight-Puzzle

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

- **States?** List of where each number is
- **Actions?** Swap the blank with an adjacent tile
- **Transitions?** Update the swapped number's new location
- **Goals?** Target configuration on the right
- **Path cost?** Number of moves

Example: Pac Man



- States?
- Actions?
- Transitions?
- Goals?
- Path cost?

Example: Pac Man



- **States?** The maze, the location of pellets, pac-man, and all ghosts
- **Actions?** Up, down, right, and left (where there isn't a wall)
- **Transitions?** Pac-man's and all ghosts' locations change
A pellet might get eaten, pac-man might die
- **Goals?** All pellets are eaten and pac-man hasn't died
- **Path cost?** Number of moves

The Value of Games in AI

- Games have well-defined rules and clear markers of success
- Game complexity can be easily adjusted without changing the underlying problem
- **Games are great at abstracting real problems**

The Value of Games in AI



Source:
https://www.theconstruct.ai/robotigniteacademy_learnros/ros-courses-library/

The Value of Games in AI

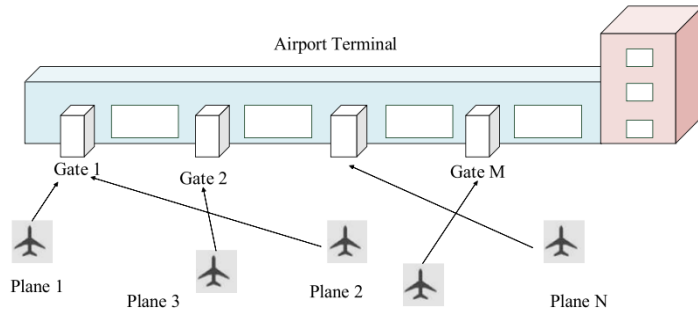
2	3	9	8	7	5	1		
4	5	6		9	2			7
	7	8	6	4				
7		1				3		8
	4	2	5	3		7		1
3			7	8		4	2	9
9	8		3		6	5	1	4
5		4	9	1			8	
6		3	4	5		9		

Source: <https://sudoku-puzzles.net/asterisk-sudoku-easy/>

[illegible]

More Examples

Airplane gate assignment



Warehouse management

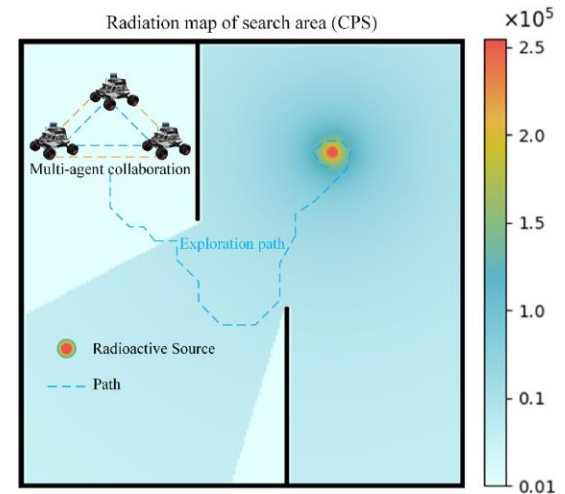


Poaching enforcement



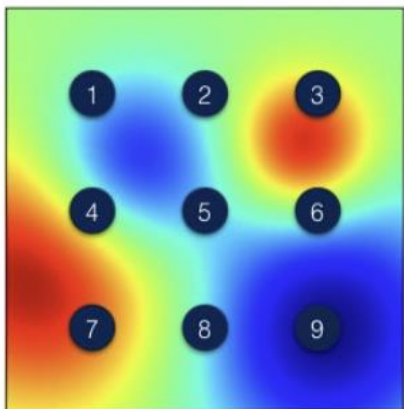
Figure 2. One Patrol Route During the Test in Uganda.

Multi-agent collaboration

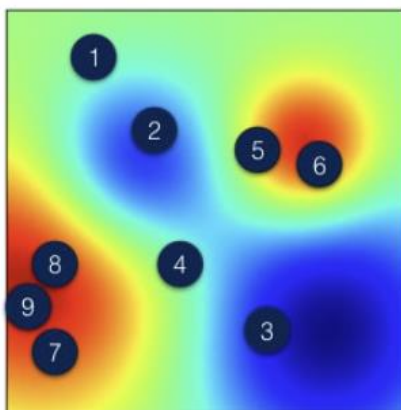


Dynamic airport gate assignment with improved Shuffled Frog-Leaping Algorithm and triangle membership function
PAWS — A Deployed Game-Theoretic Application to Combat Poaching
<https://www.linkedin.com/pulse/warehousing-inventory-management-critical-topics-within-meshack-sofof>
PSOPF-MATD3: A multi-agent collaborative radioactive source search strategy

More Examples



Grid Search



Adaptive Selection

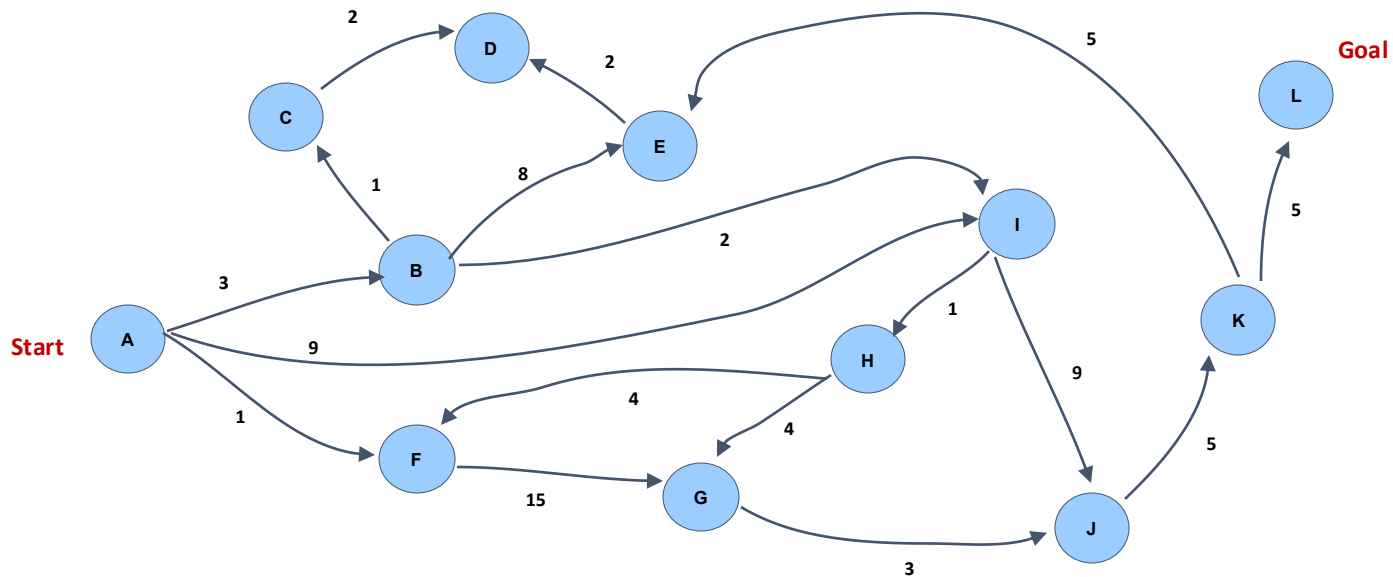
Hyperparameter optimization

```
1 void zunebug_repair(int days) {
2   int year = 1980;
3   while (days > 365) {
4     if (isLeapYear(year)) {
5       if (days > 366) {
6         // days -= 366; // repair deletes
7         year += 1;
8       }
9     } else {
10      }
11     days -= 366;      // repair inserts
12   } else {
13     days -= 365;
14     year += 1;
15   }
16 }
17 printf("current year is %d\n", year);
18 }
```

Automated program repair

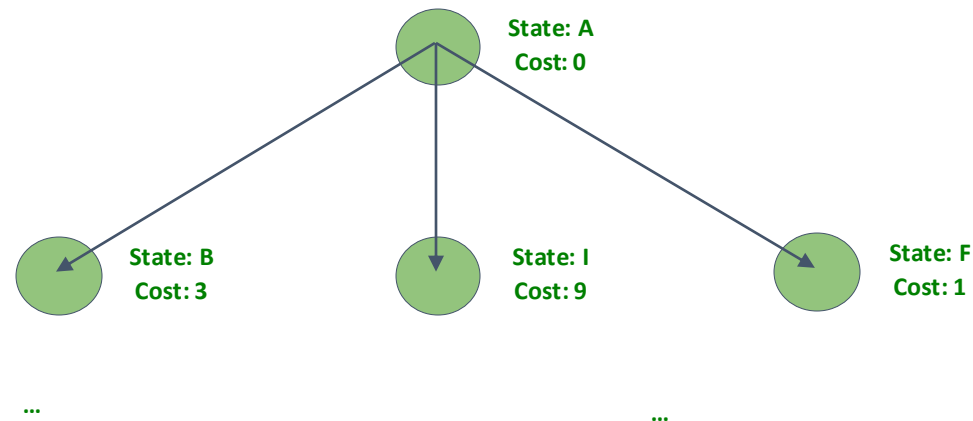
A Common Representation: State Space Graphs

- Graph defined by a set of vertices and edges where:
 - Vertices correspond to states in **S**
 - Edges indicate that some operator allows transitioning between states

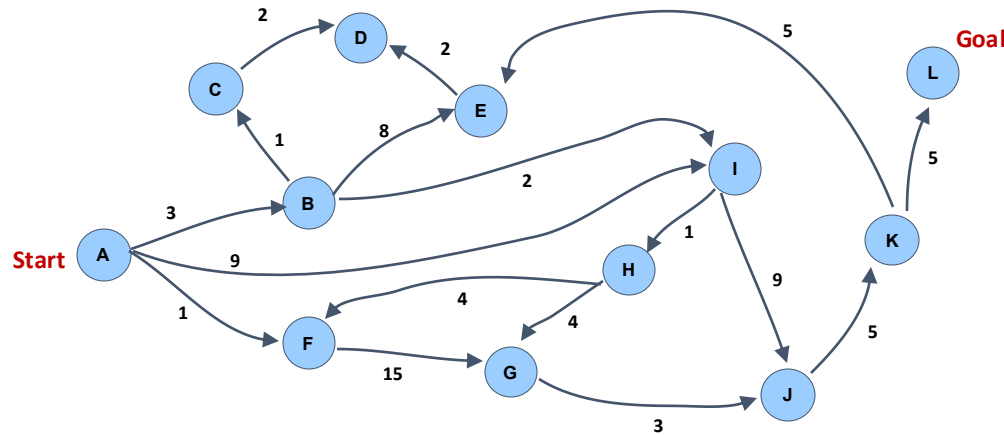


Search Trees

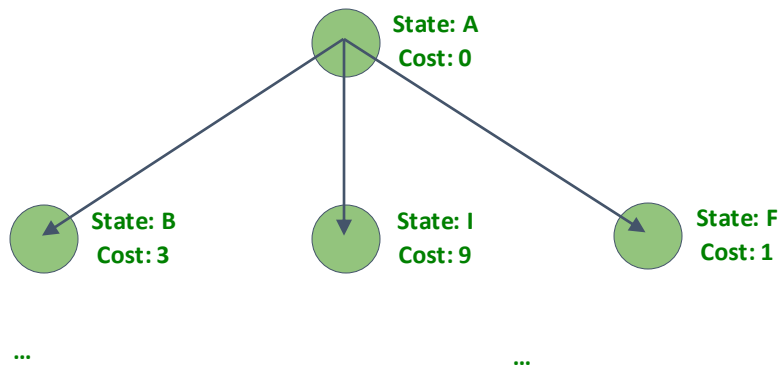
- Represents the exploration paths in a search procedure
 - Nodes represent current progress, must store latest state in **S** and any extra info required by the search method
 - Edges correspond to operators
- We want to search only until a goal is found, and then retrace our steps to return the path



State Space Graphs vs Search Trees



State space
graph



Sample search
tree

Questions?

Uninformed Search

- Goal states have binary “goal” information.
- Non-goal states have **no information** about distance to goal.
- **Hence, all you can do is move systematically between states until you stumble on a goal.**

Breadth-First Search vs Depth-First Search

Breadth-First Search vs Depth-First Search

BFS

“siblings over children”

DFS

“children over siblings”

Breadth-First Search vs Depth-First Search

BFS

“siblings over children”



DFS

“children over siblings”



initial node

Breadth-First Search vs Depth-First Search

BFS

DFS

“siblings over children”

“children over siblings”

Round 1

initial node
unexpanded nodes

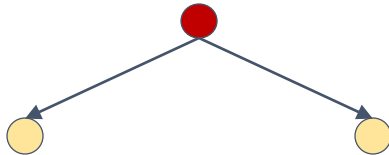


Breadth-First Search vs Depth-First Search

BFS

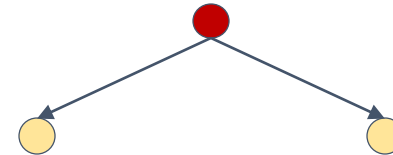
“siblings over children”

Round 2



DFS

“children over siblings”



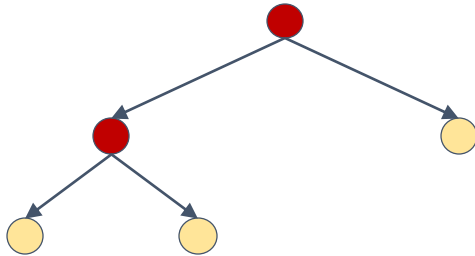
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

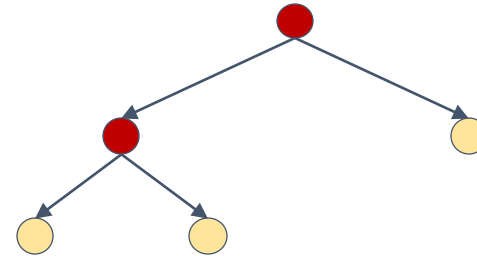
“siblings over children”

Round 3



DFS

“children over siblings”



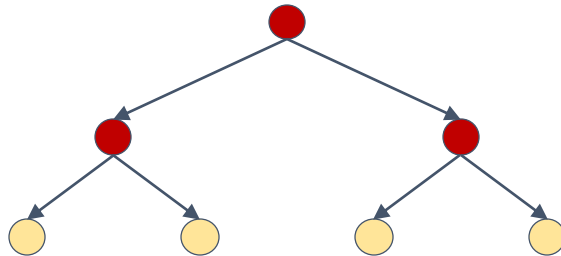
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

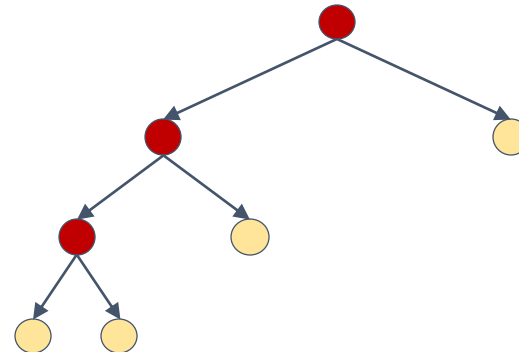
“siblings over children”

Round 4



DFS

“children over siblings”



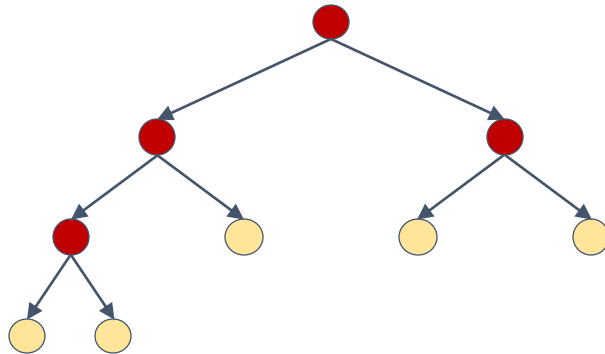
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

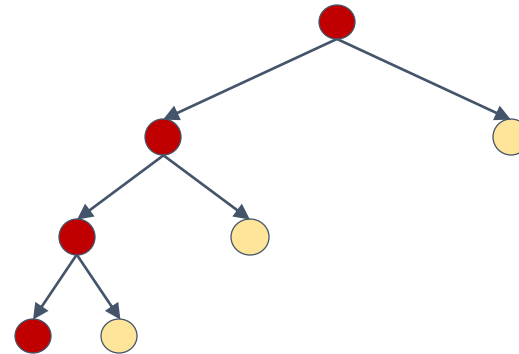
“siblings over children”

Round 5



DFS

“children over siblings”



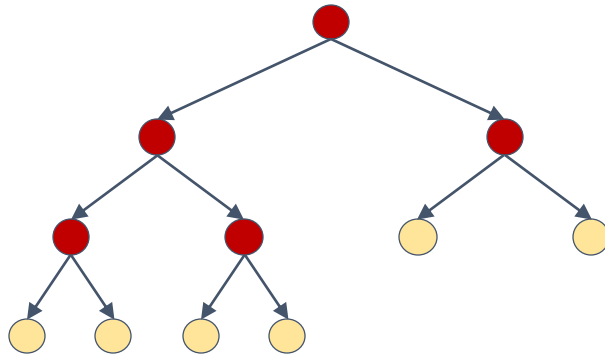
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

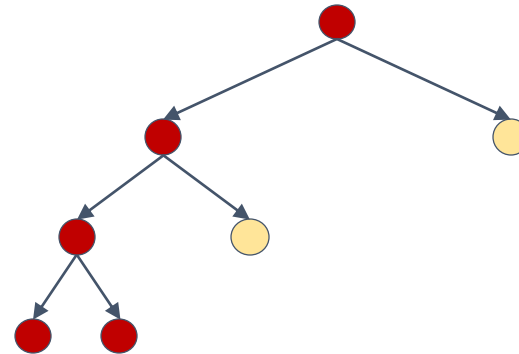
“siblings over children”

Round 6



DFS

“children over siblings”



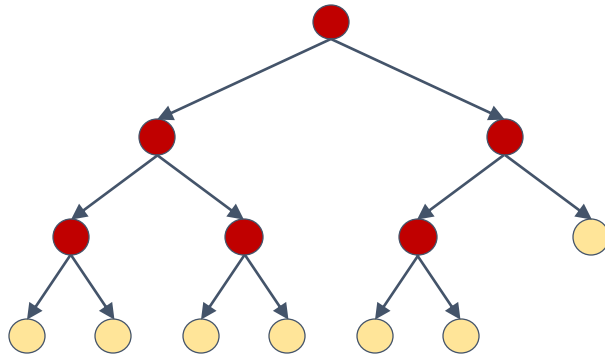
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

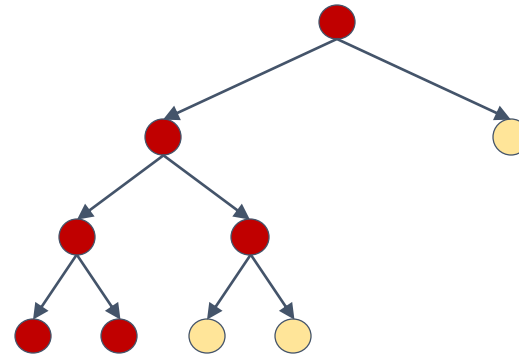
“siblings over children”

Round 7



DFS

“children over siblings”



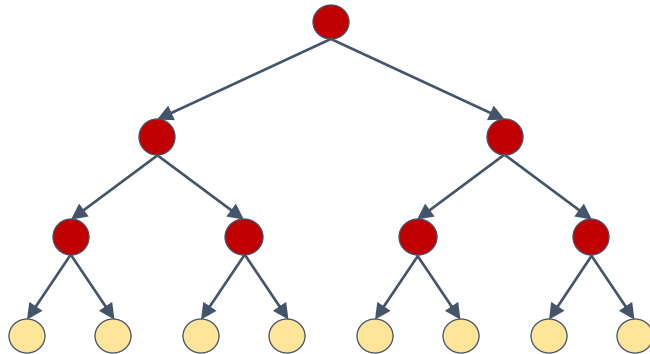
initial node
unexpanded nodes
expanded nodes

Breadth-First Search vs Depth-First Search

BFS

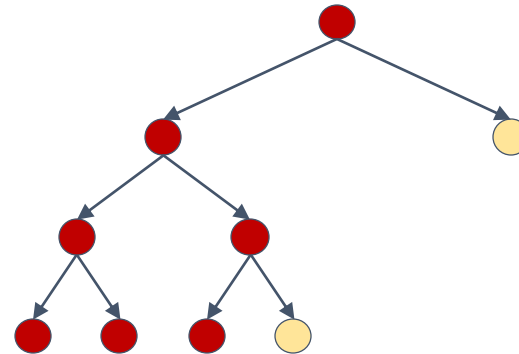
“siblings over children”

Round 8



DFS

“children over siblings”

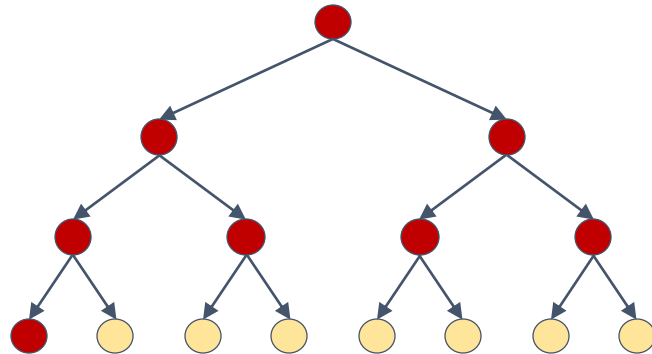


Breadth-First Search vs Depth-First Search

BFS

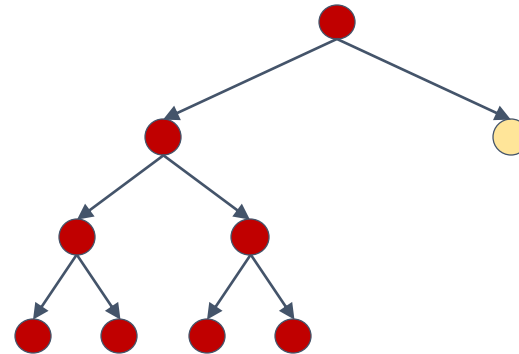
“siblings over children”

Round 9



DFS

“children over siblings”



Breadth-First Search vs Depth-First Search

BFS

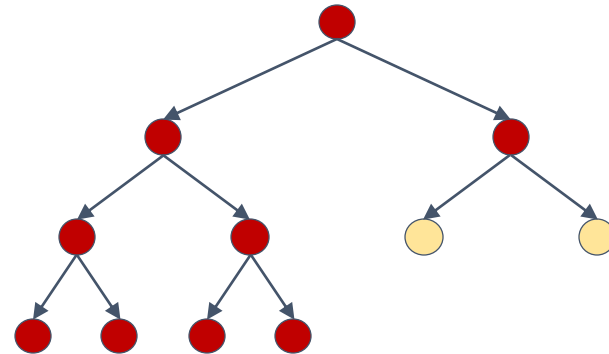
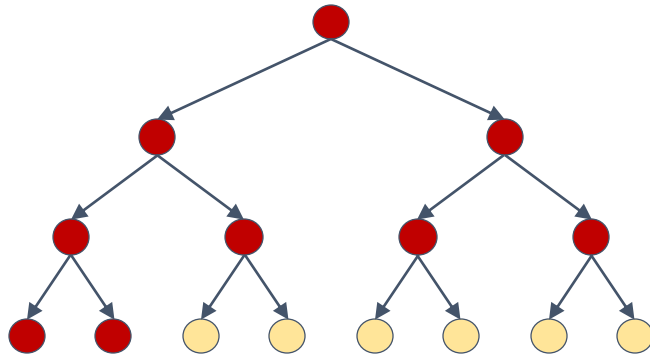
“siblings over children”

DFS

“children over siblings”

Round 10

initial node
unexpanded nodes
expanded nodes



Breadth-First Search vs Depth-First Search

BFS

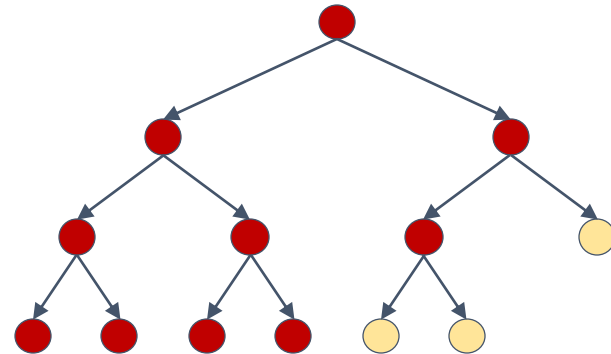
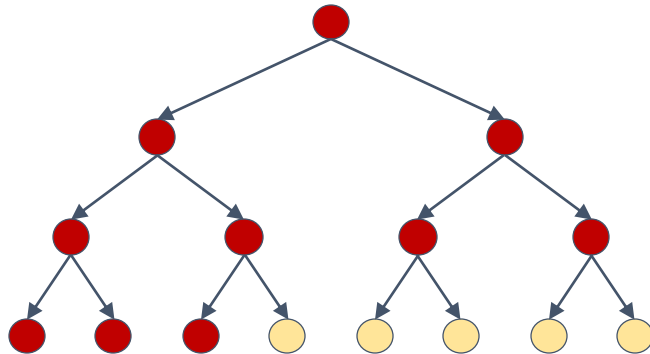
“siblings over children”

DFS

“children over siblings”

Round 11

initial node
unexpanded nodes
expanded nodes



Breadth-First Search vs Depth-First Search

BFS

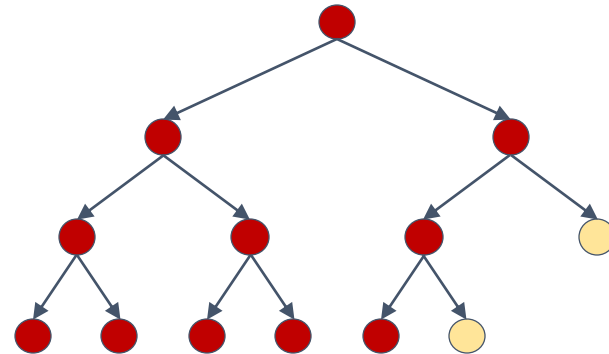
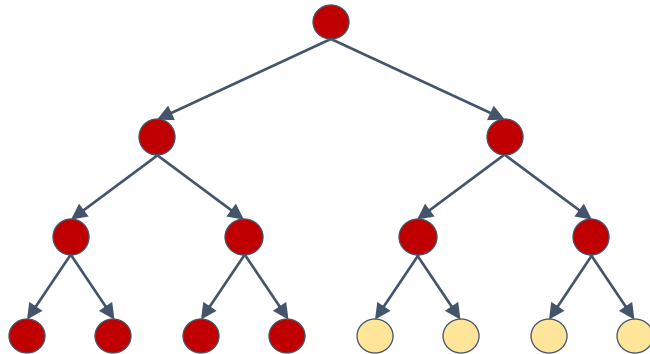
“siblings over children”

DFS

“children over siblings”

Round 12

initial node
unexpanded nodes
expanded nodes



Breadth-First Search vs Depth-First Search

BFS

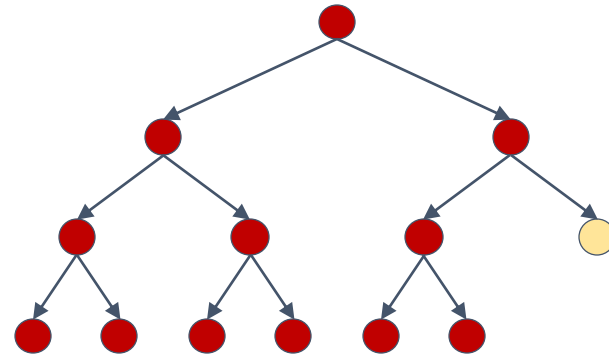
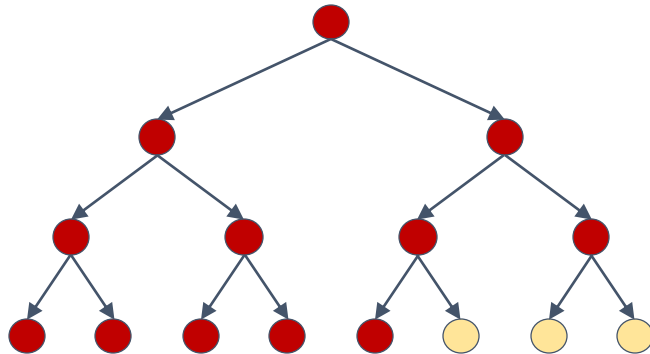
“siblings over children”

DFS

“children over siblings”

Round 13

initial node
unexpanded nodes
expanded nodes



Breadth-First Search vs Depth-First Search

BFS

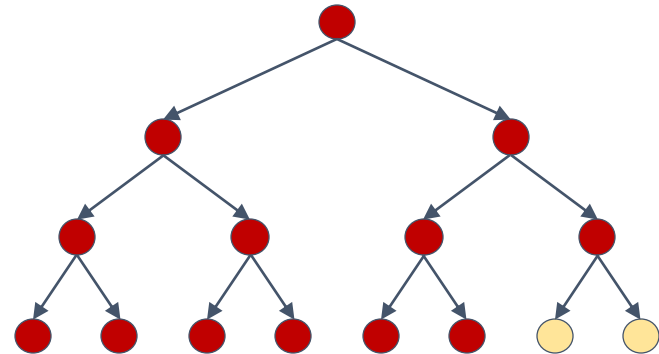
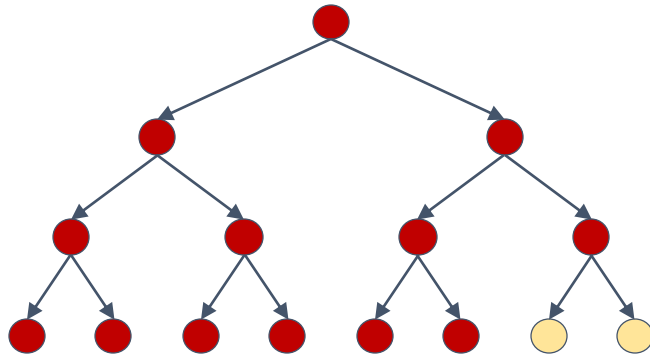
“siblings over children”

DFS

“children over siblings”

Round 14

initial node
unexpanded nodes
expanded nodes



Breadth-First Search vs Depth-First Search

BFS

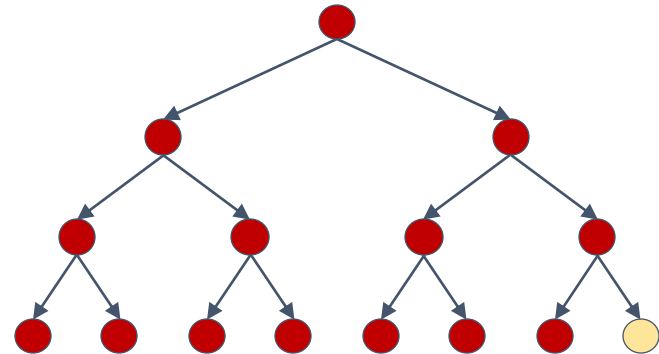
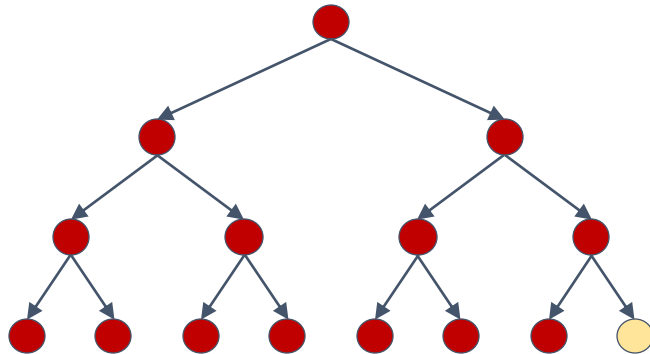
“siblings over children”

DFS

“children over siblings”

initial node
unexpanded nodes
expanded nodes

Round 15



Breadth-First Search vs Depth-First Search

BFS

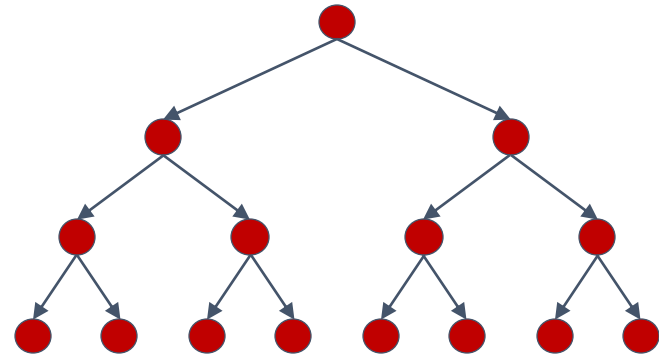
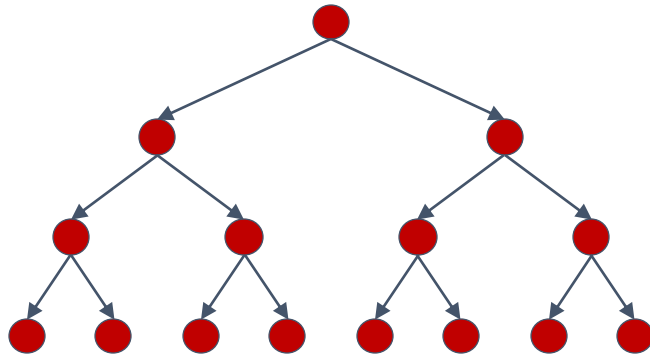
“siblings over children”

DFS

“children over siblings”

Round 16

initial node
unexpanded nodes
expanded nodes



Key Properties of an Algorithm

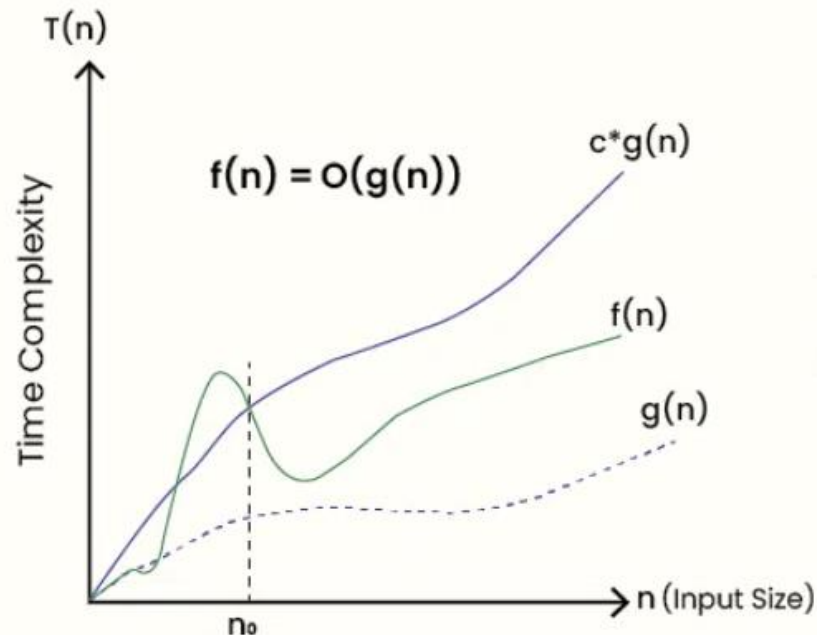
- **Completeness:** Are we assured to find a solution, if one exists?
- **Optimality:** How good is the solution?
- **Space complexity:** How much storage is needed?
- **Time complexity:** How many operations are needed?

Key Properties of an Algorithm

- **Completeness:** Are we assured to find a solution, if one exists?
- **Optimality:** How good is the solution?
- **Space complexity:** How much storage is needed?
- **Time complexity:** How many operations are needed?
- Typically conditioned on:
 - The precise implementation of the search method (do we check duplicates? Early/late goal check?)
 - Properties of the search graph (contains loops? (in)finite? goal depth?)

Another Detour: Big O Notation

- How long does code run, or how much memory does it consume?
- Big O is about finding an **asymptotic upper bound**.
 - i.e., how an algorithm's runtime or spacetime **scales** as input grows
- We want a summary of the dominant scaling factor:
 - $f(n) = O(g(n))$ if there exist positive constants c, n_0 s.t. $f(n) \leq c * g(n)$ for all $n \geq n_0$



Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

BFS

DFS

Completeness
(finite “ b ”)

Optimality
(step costs=1)

Space
Complexity

Time
Complexity

Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

BFS

DFS

Completeness
(finite “b”)

Optimality
(step costs=1)

Space
Complexity

Time
Complexity

$O(b^d)$

$O(b^m)$

Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

BFS

DFS

Completeness
(finite “b”)

Optimality
(step costs=1)

Space
Complexity

$O(b^d)$

$O(bm)$

Time
Complexity

$O(b^d)$

$O(b^m)$

Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

	BFS	DFS
Completeness (finite “b”)	Yes	No
Optimality (step costs=1)		
Space Complexity	$O(b^d)$	$O(bm)$
Time Complexity	$O(b^d)$	$O(b^m)$

Key Properties

$b \Rightarrow$ branching factor

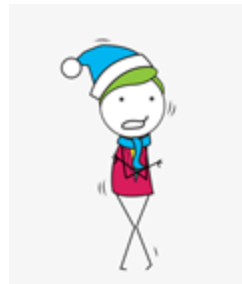
$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

	BFS	DFS
Completeness (finite “b”)	Yes	No
Optimality (step costs=1)	Yes	No
Space Complexity	$O(b^d)$	$O(bm)$
Time Complexity	$O(b^d)$	$O(b^m)$

General Step Cost

- Actions often have different costs
- e.g., I can buy a burger for \$5 or fries for \$3
- e.g., I can walk to the classroom outside at a cost of 500 units of warmth, or I can walk through the underground tunnels at a cost of 5 units of warmth



Uniform Cost Search

- Goal: Fix BFS to ensure an optimal path with **general** step costs (i.e., in a *weighted* graph).

Important distinction:

- **Unit cost** = Problem where each action has the same cost.
- **General cost** = Actions can have different costs.

Uniform Cost Search

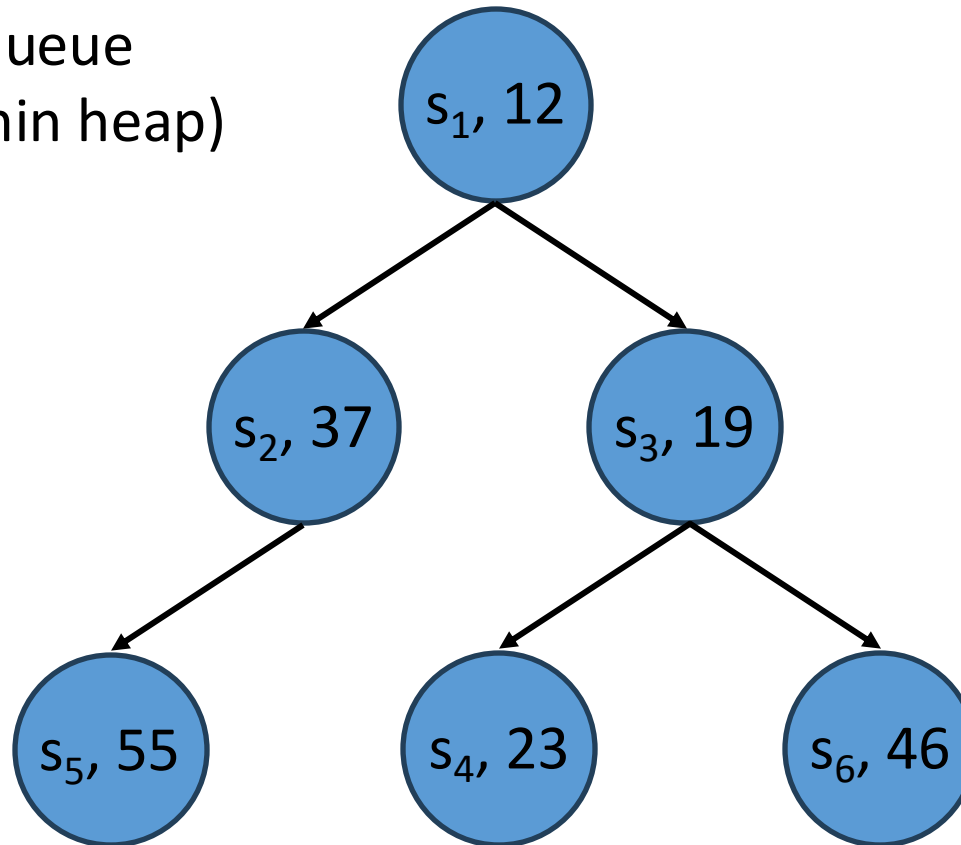
- Goal: Fix BFS to ensure an optimal path with **general** step costs (i.e., in a *weighted* graph).

Important distinction:

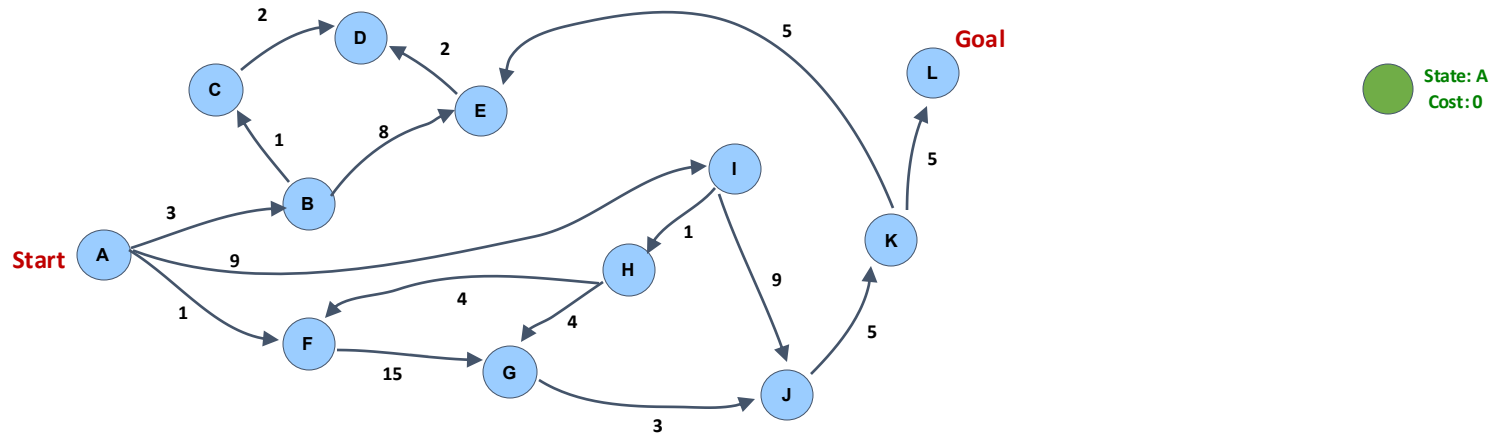
- **Unit cost** = Problem where each action has the same cost.
- **General cost** = Actions can have different costs.
- Approach:
 - Use a **priority queue** instead of a simple queue.
 - Insert nodes in the increasing order of the cost of the path so far.
- Properties:
 - Guaranteed to find **optimal solution** for graph with general step costs (same as BFS when all operators have the same cost).

Uniform Cost Search

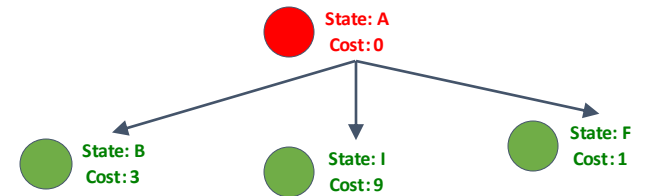
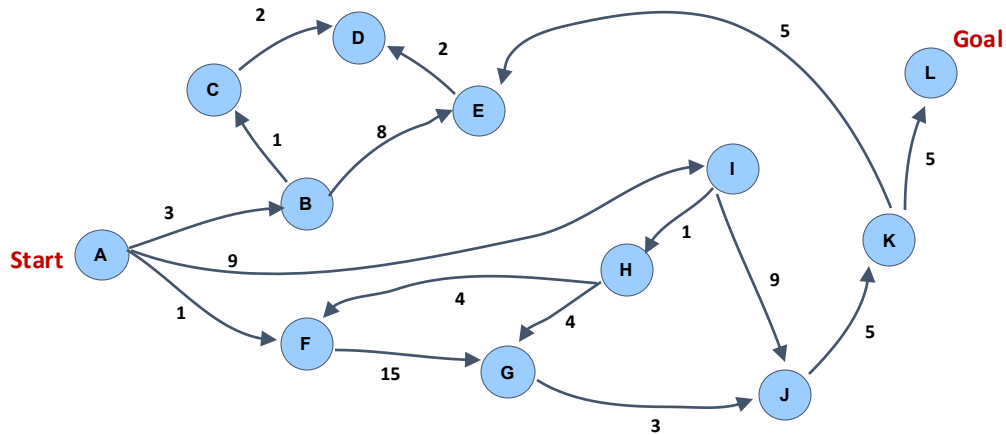
Priority queue
(here a min heap)



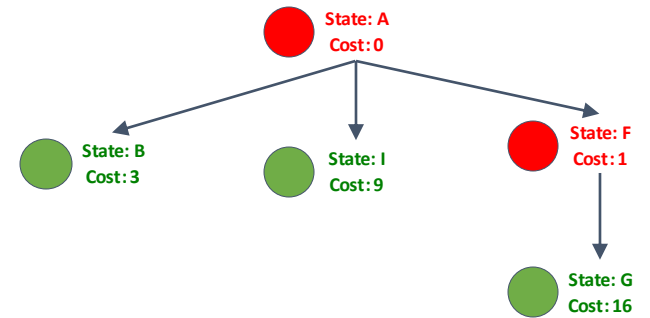
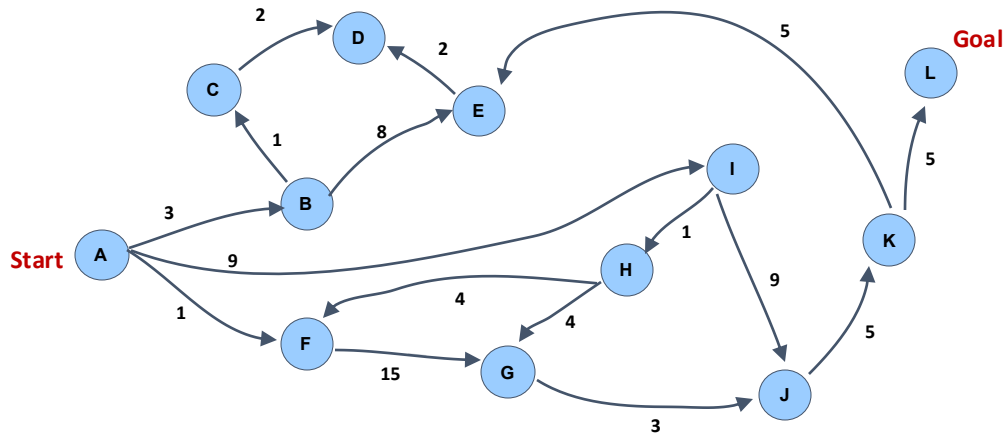
Example: Shortest Path



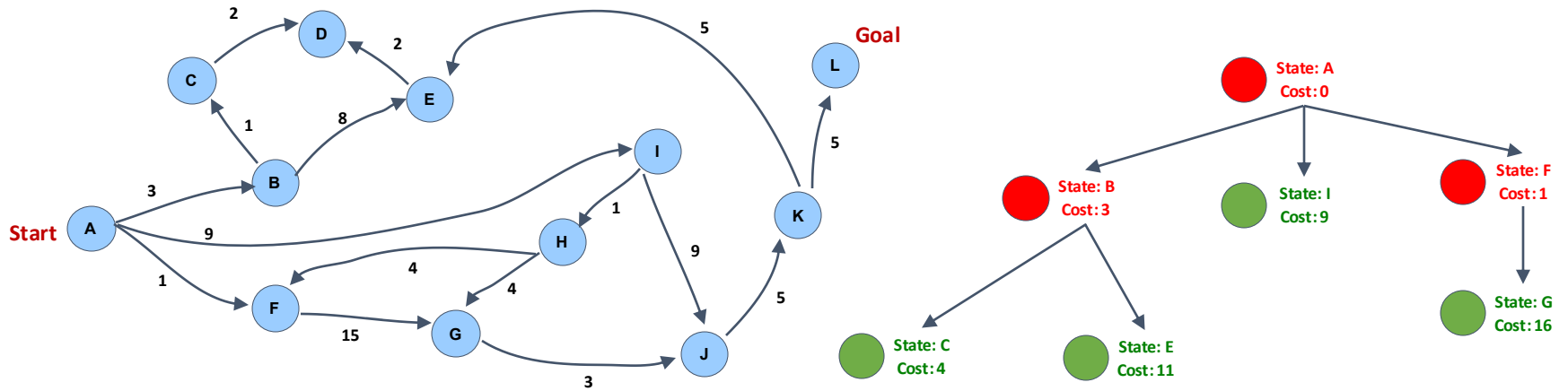
Example: Shortest Path



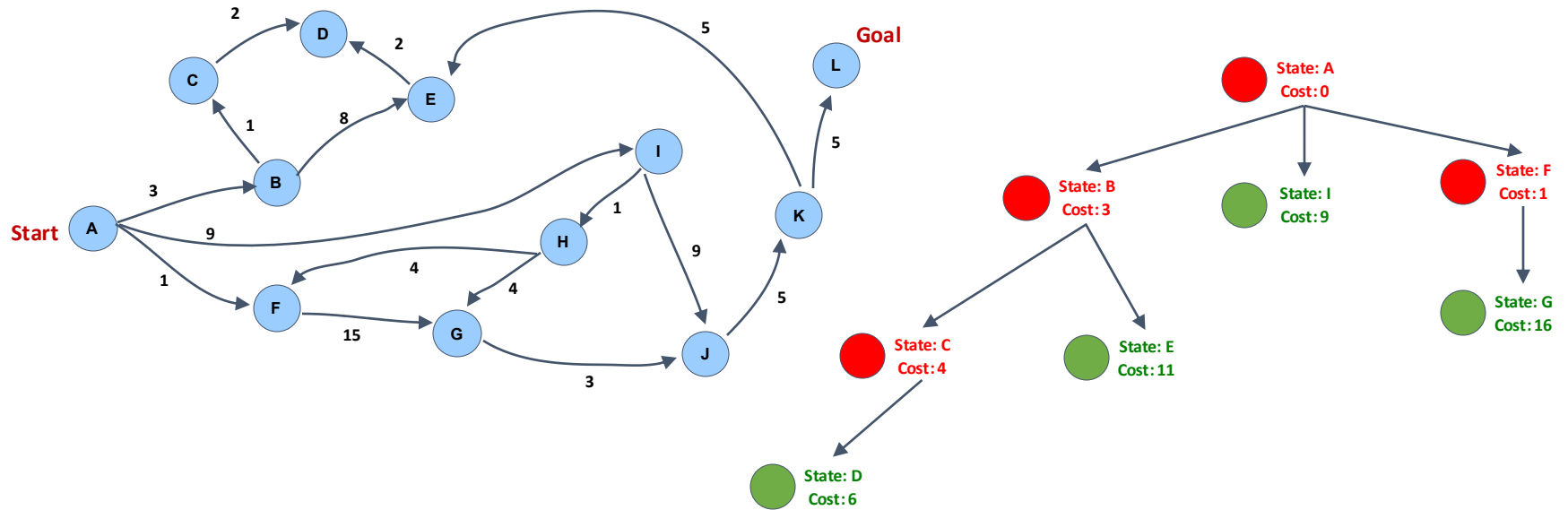
Example: Shortest Path



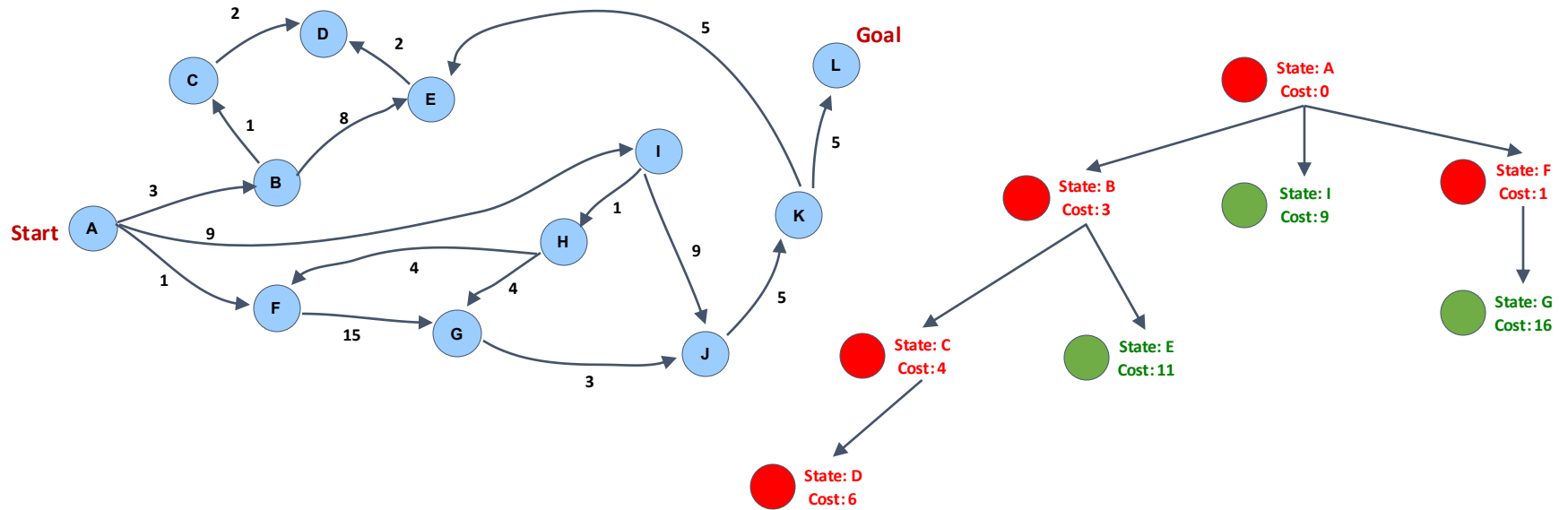
Example: Shortest Path



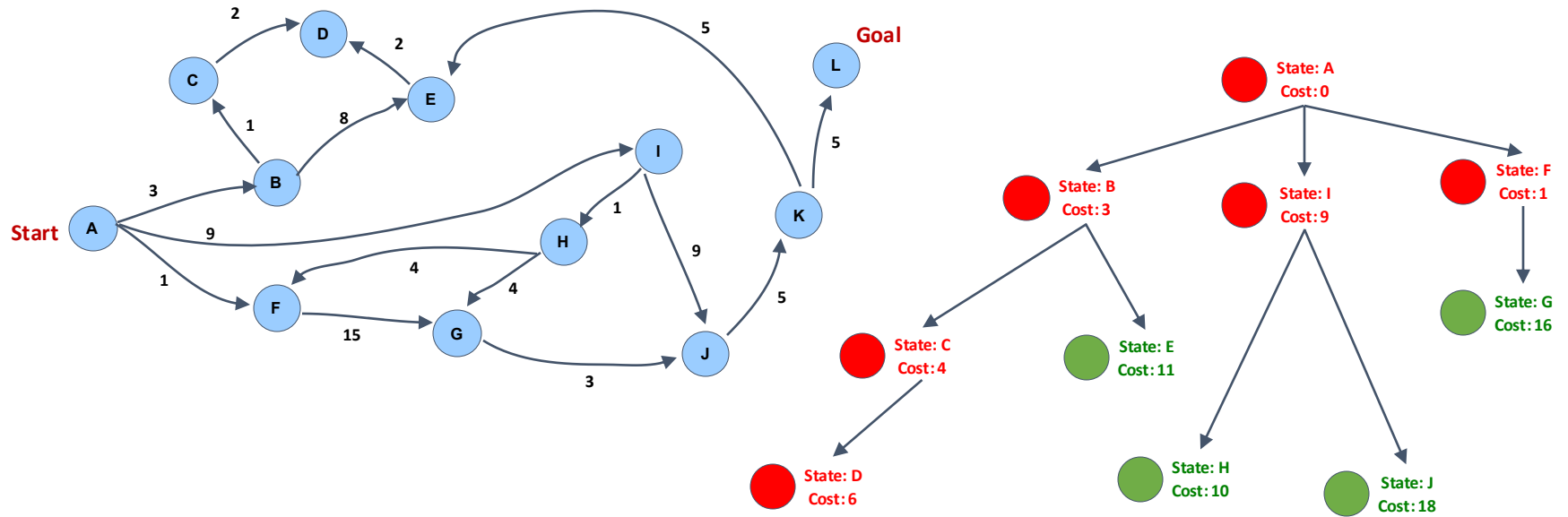
Example: Shortest Path



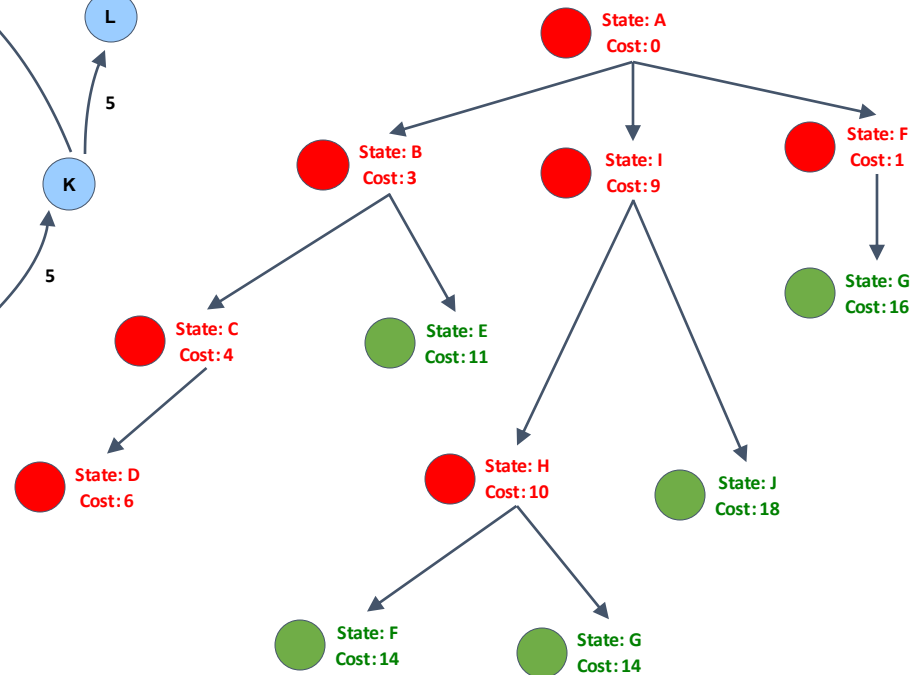
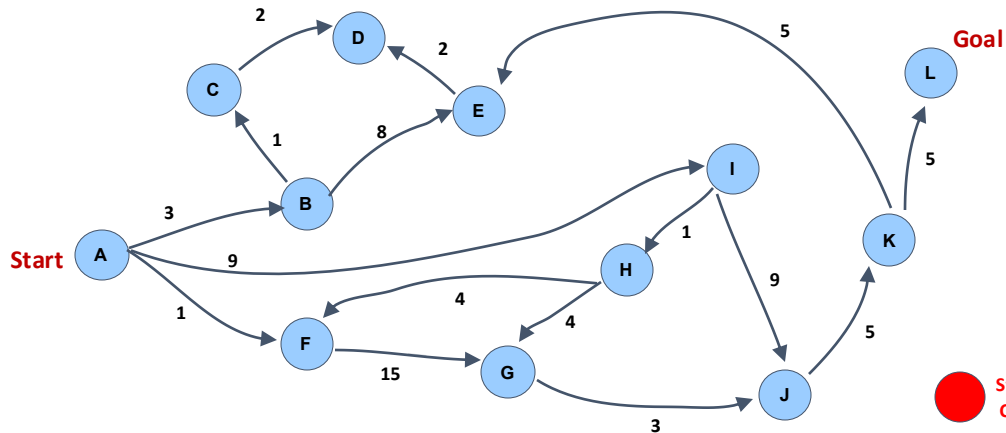
Example: Shortest Path



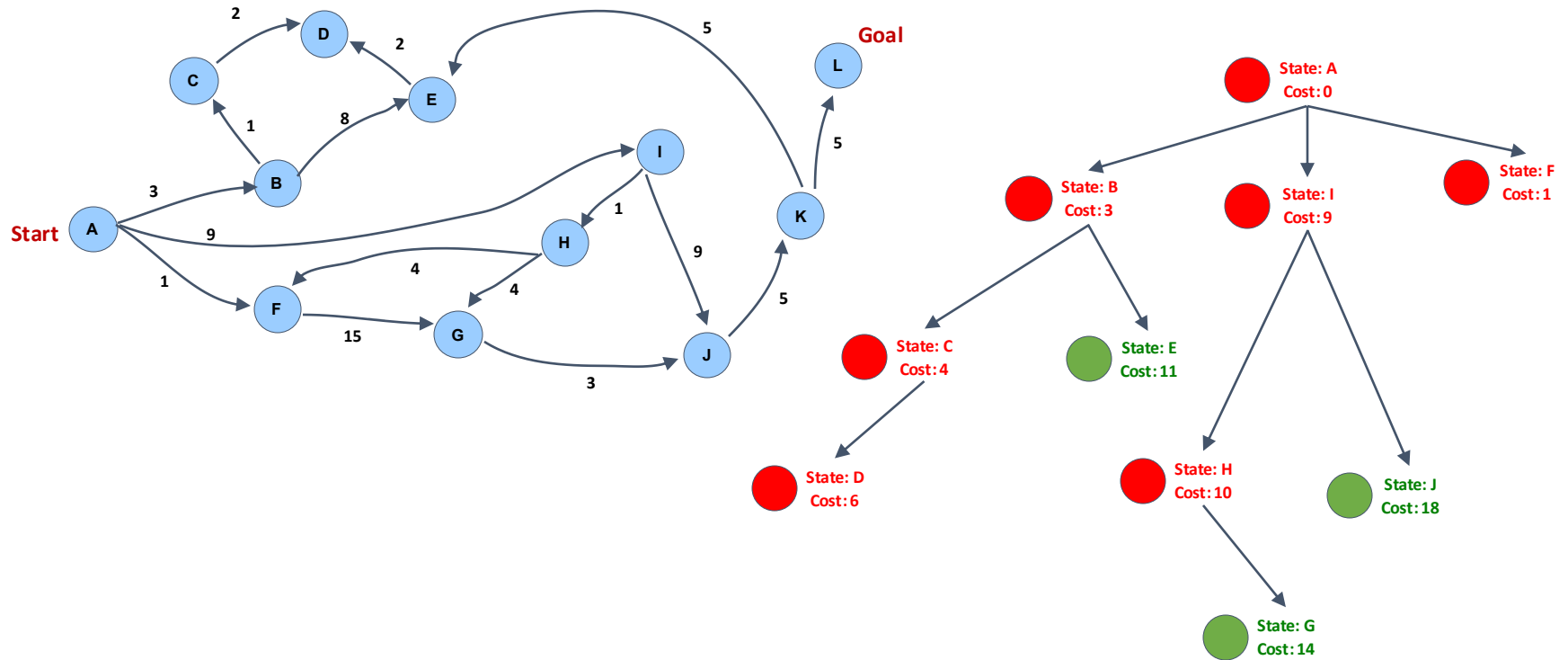
Example: Shortest Path



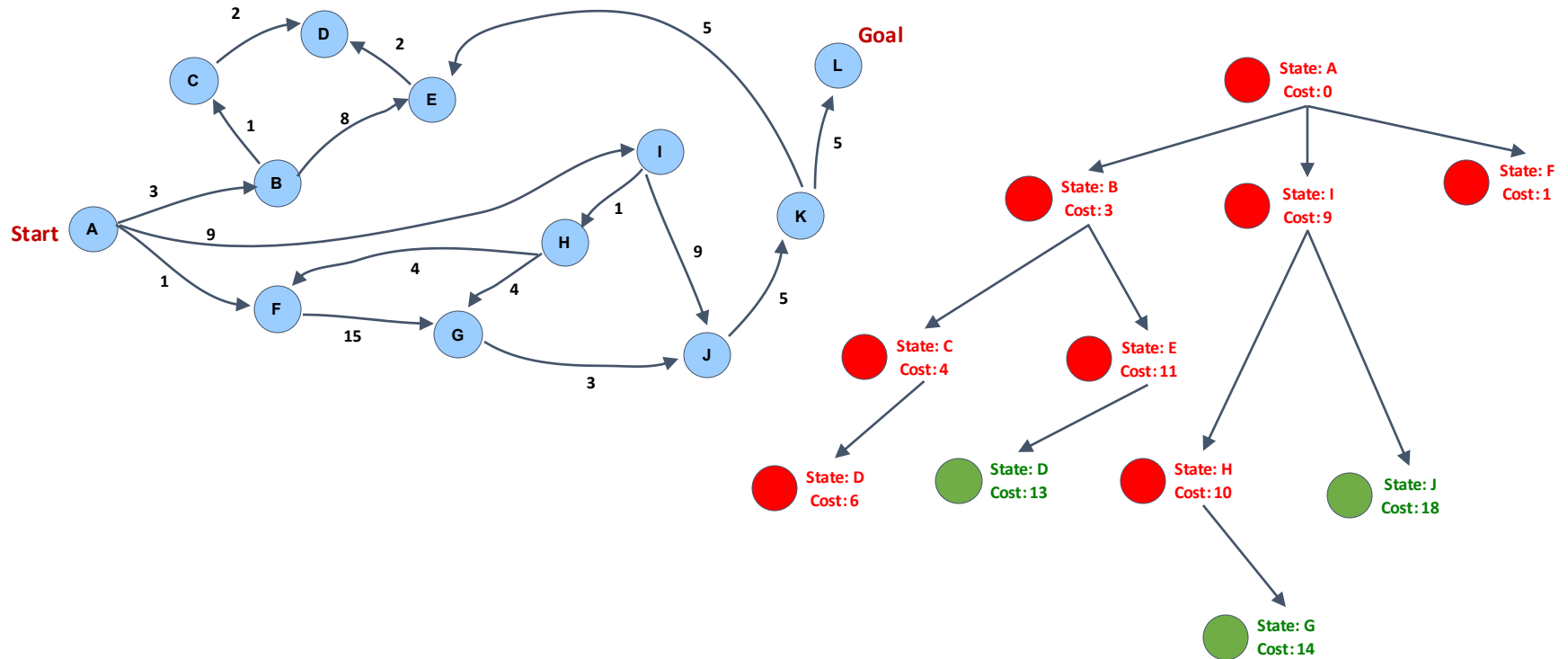
Example: Shortest Path



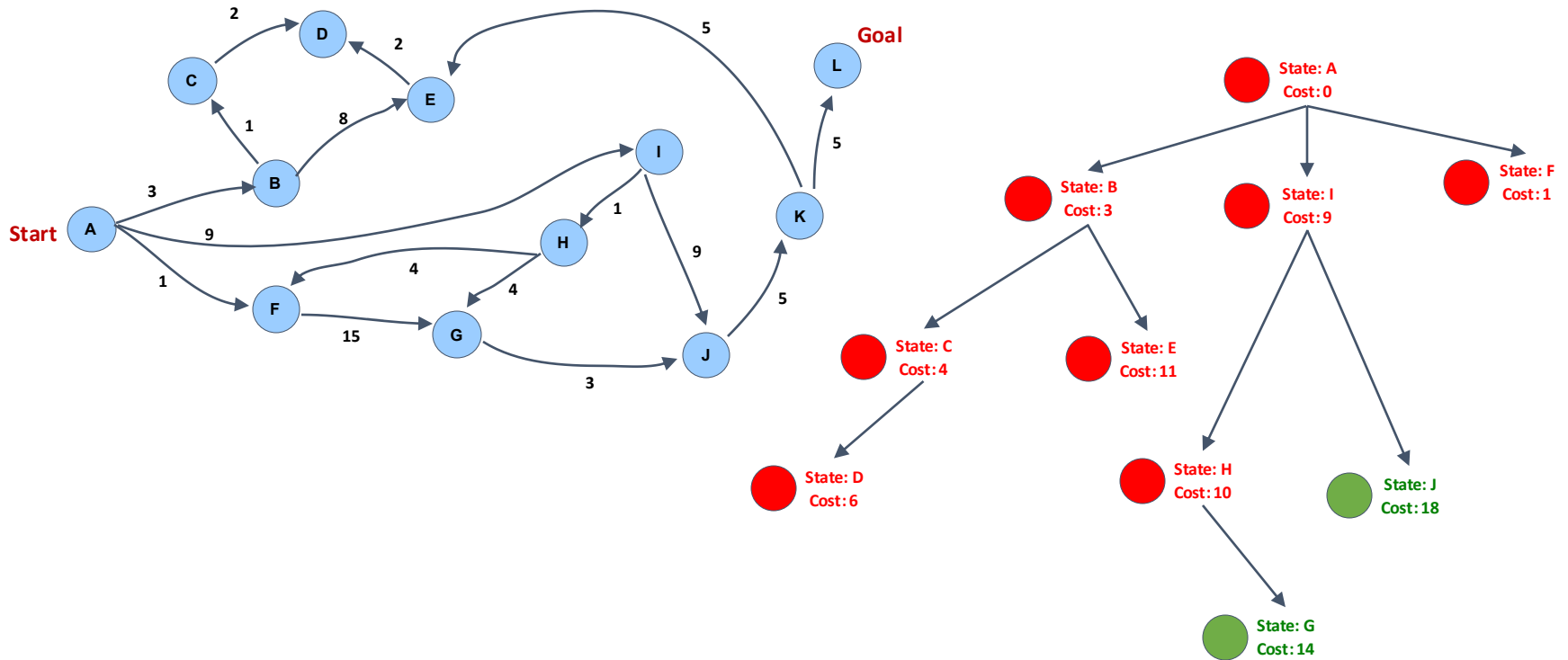
Example: Shortest Path



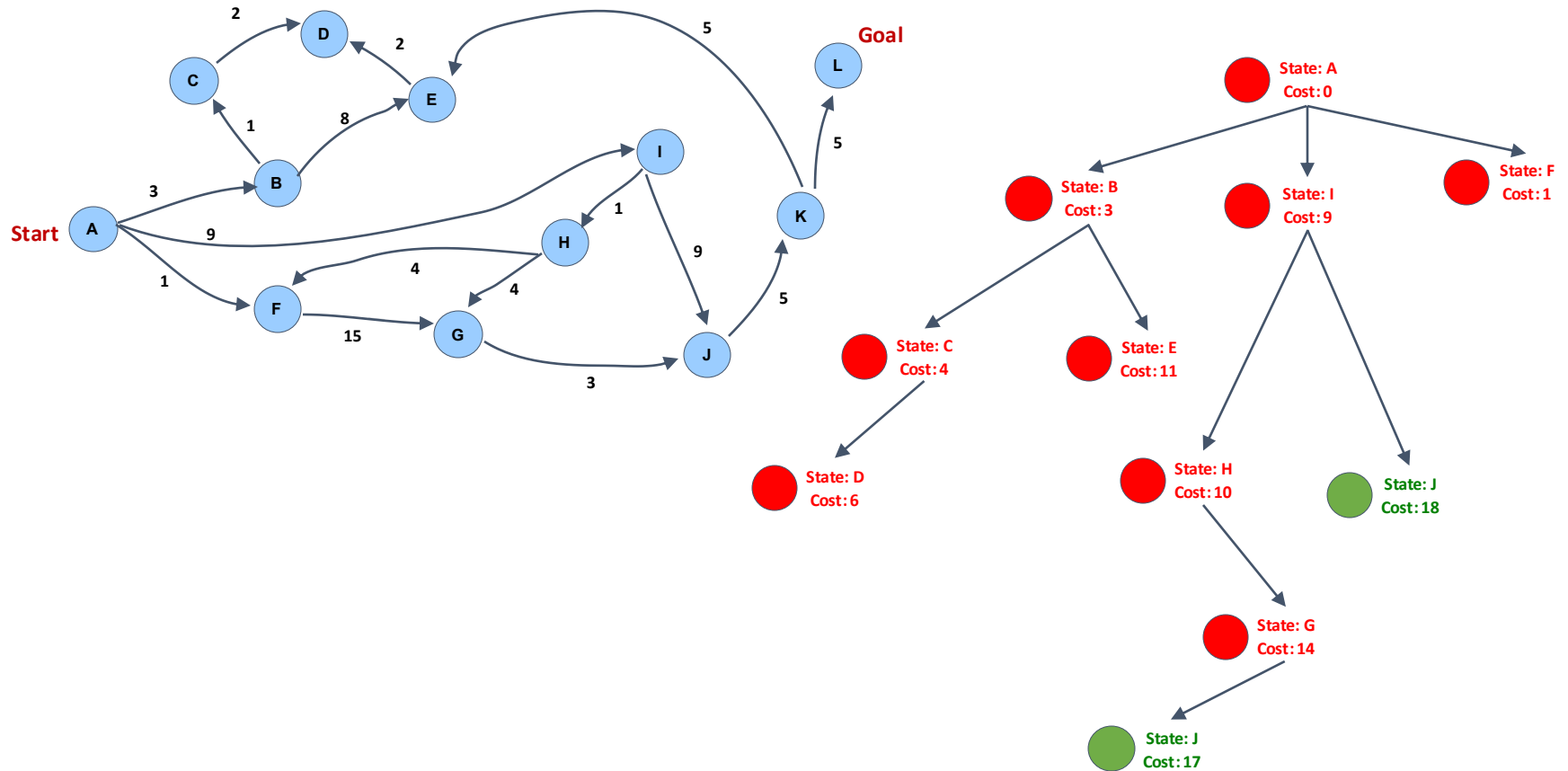
Example: Shortest Path



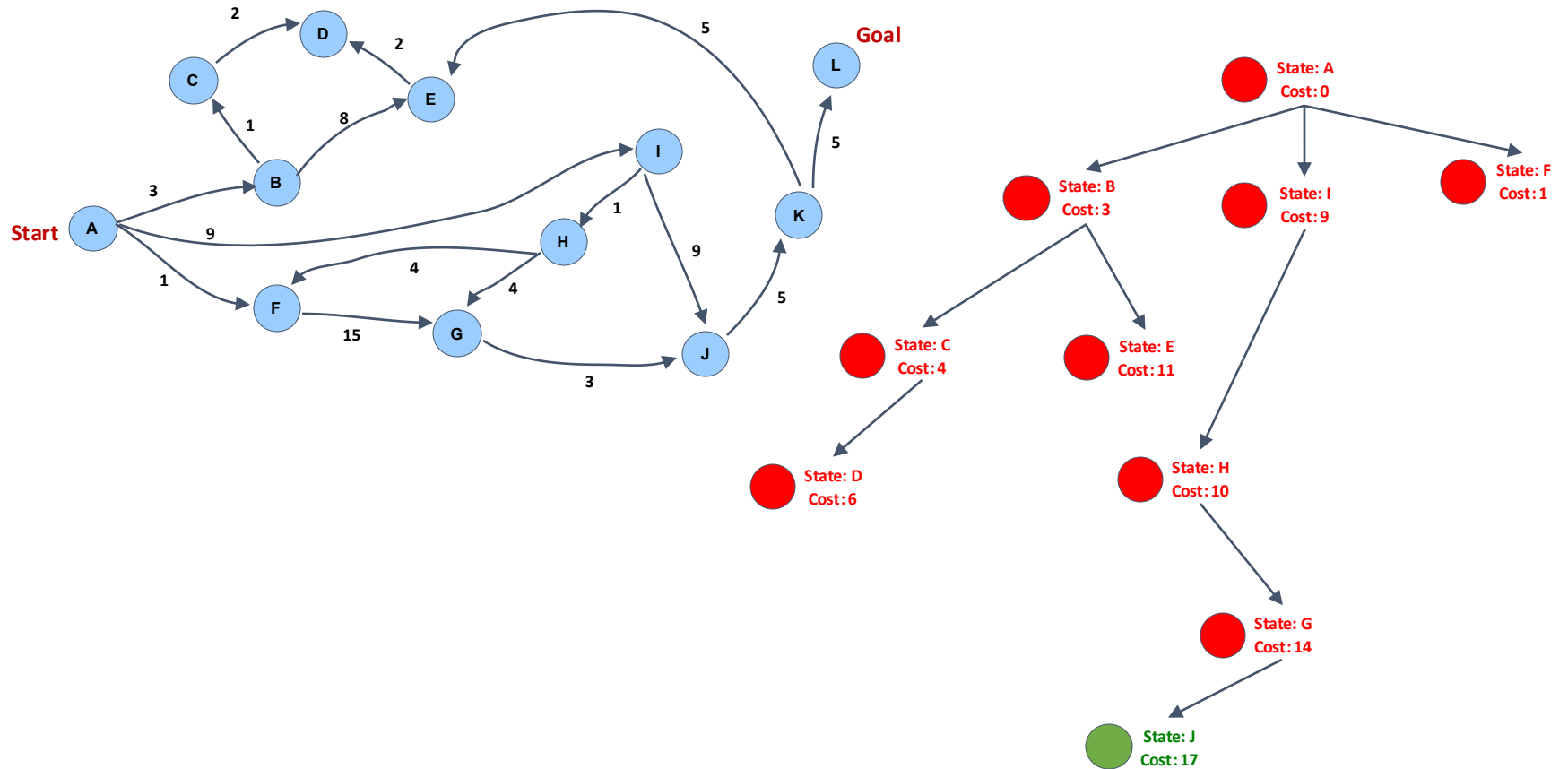
Example: Shortest Path



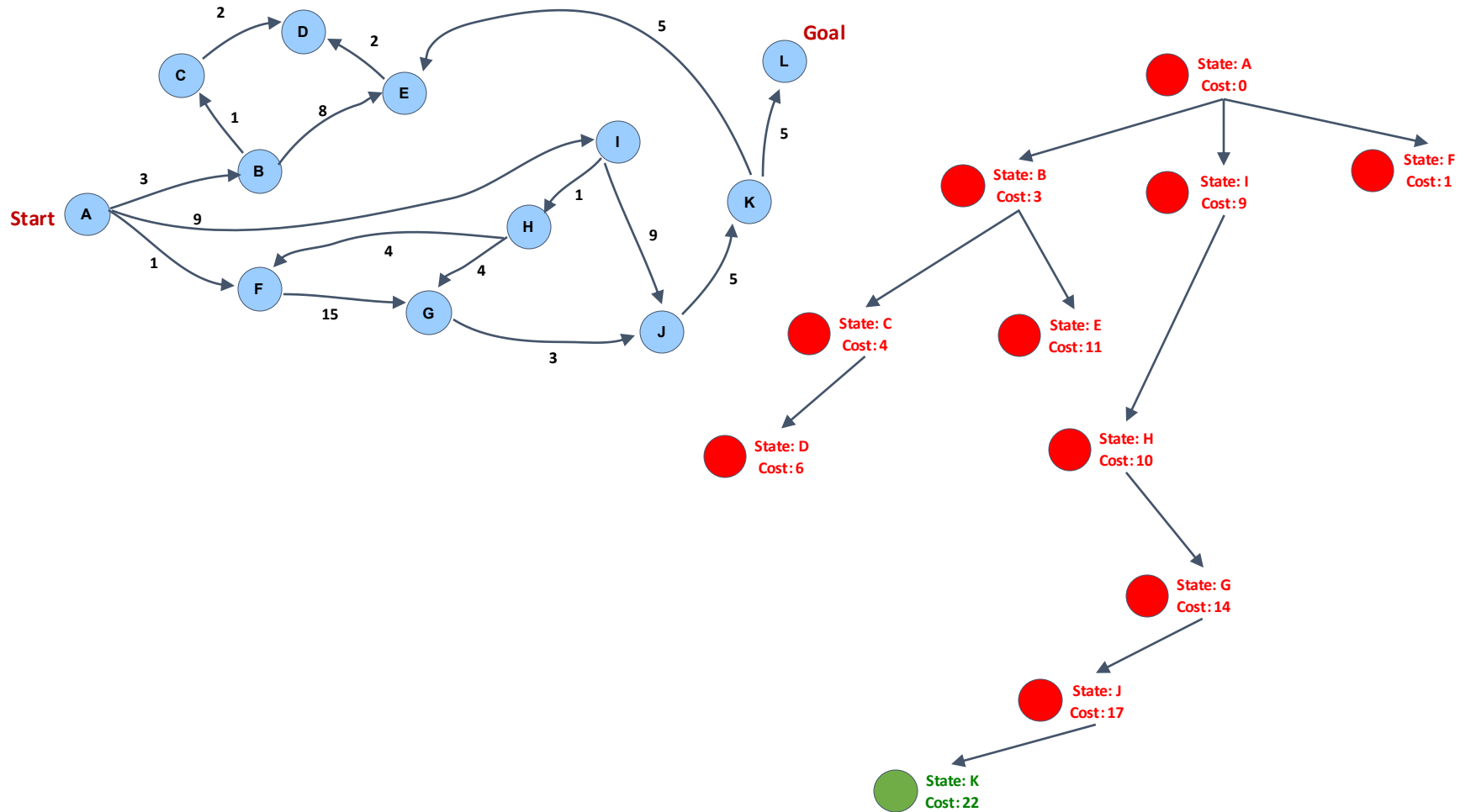
Example: Shortest Path



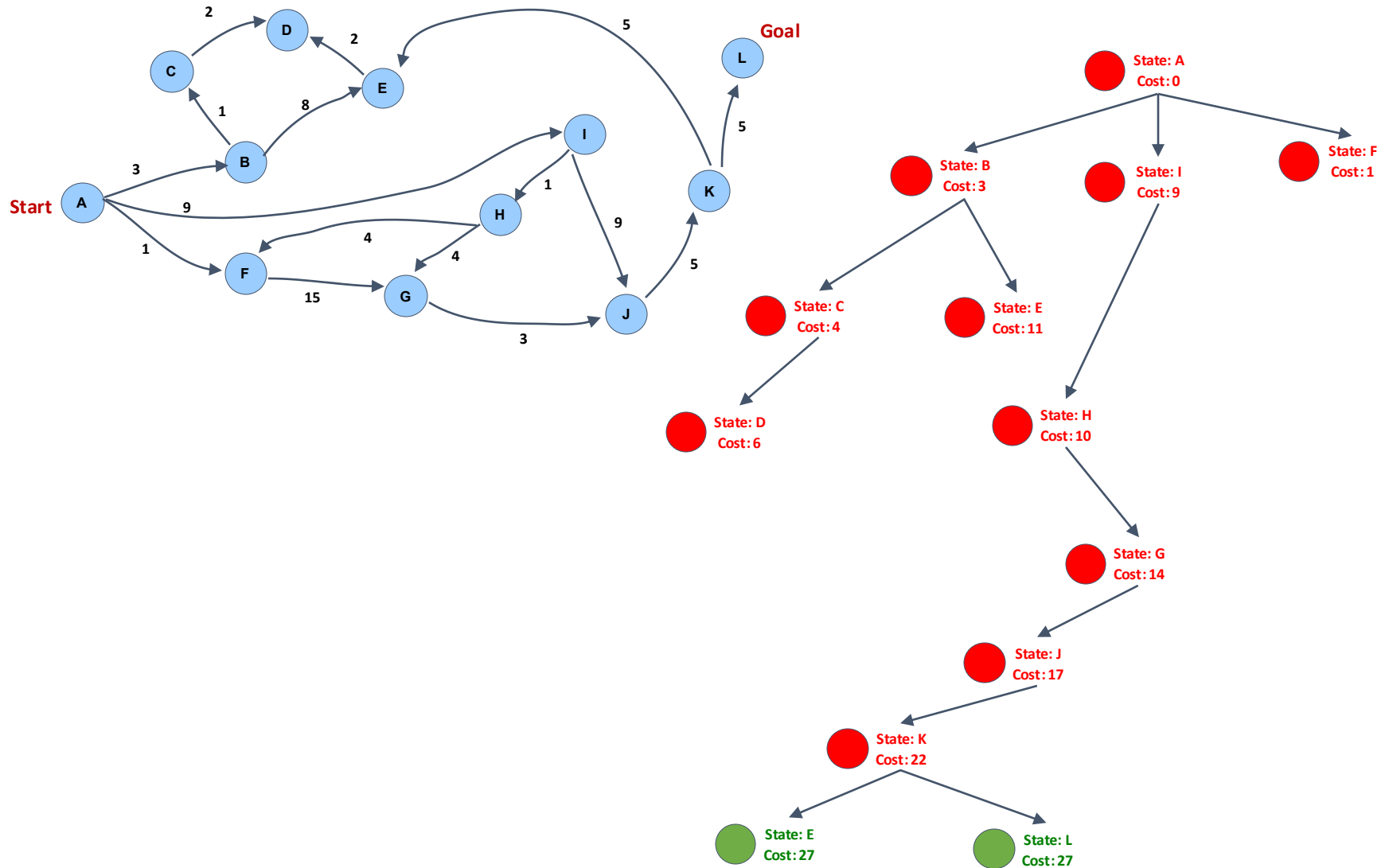
Example: Shortest Path



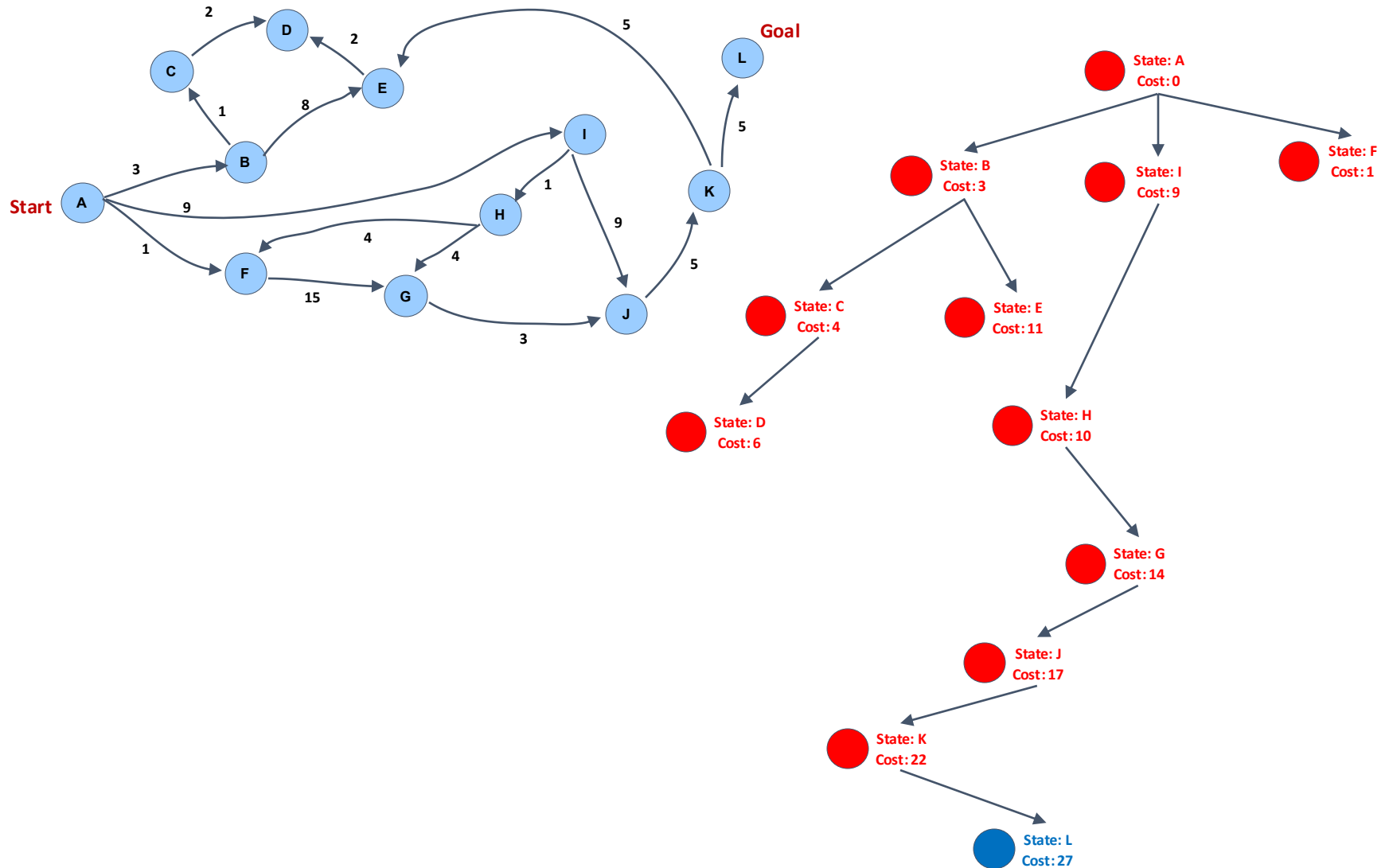
Example: Shortest Path



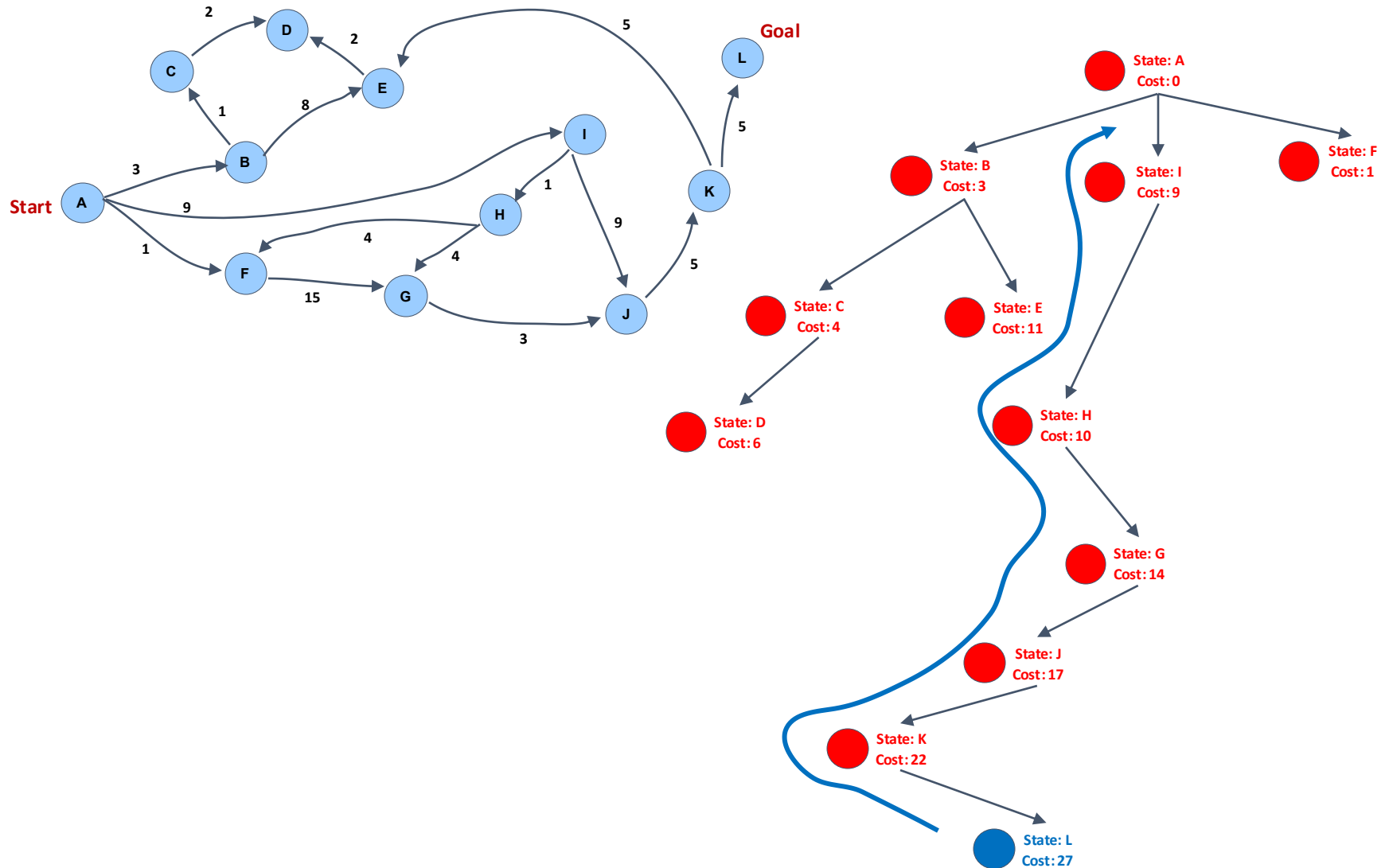
Example: Shortest Path



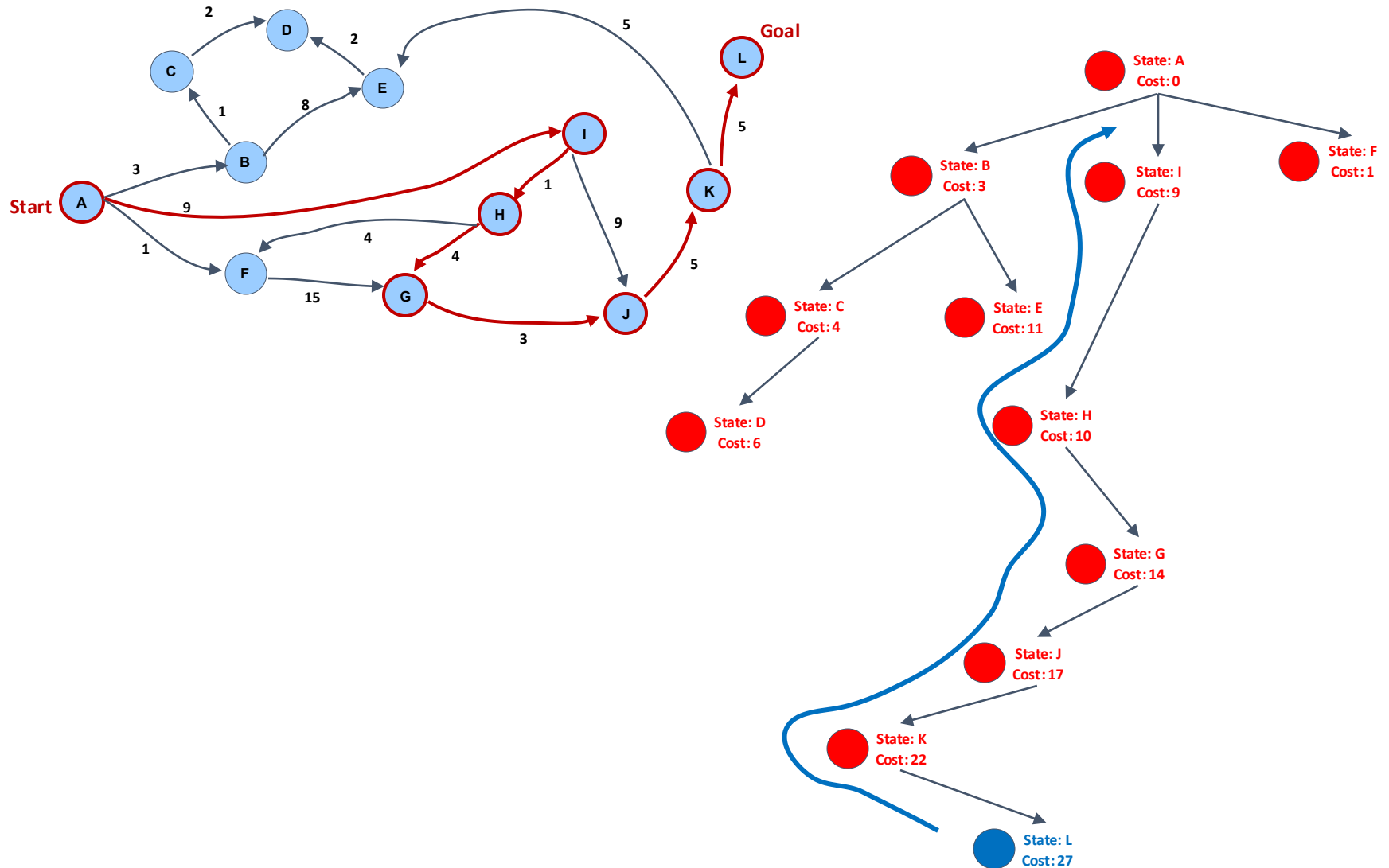
Example: Shortest Path



Example: Shortest Path



Example: Shortest Path



Key Properties of an Algorithm

- **Completeness:** Are we assured to find a solution, if one exists?
- **Optimality:** How good is the solution?
- **Space complexity:** How much storage is needed?
- **Time complexity:** How many operations are needed?

Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

$\epsilon \Rightarrow$ minimum (positive) edge cost

$C^* \Rightarrow$ optimal path cost

	BFS	DFS	Uniform
Completeness (finite “b”)	Yes	No	Yes*
Optimality (step costs=1)	Yes	No	Yes
Optimality (general costs)	No	No	Yes
Space Complexity	$O(b^d)$	$O(bm)$	$O(b^{C^*/\epsilon})$
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^{C^*/\epsilon})$

Depth-Limited Search

Depth-first search, but terminate a path either if a goal state is found, or if the maximum depth allowed is reached.

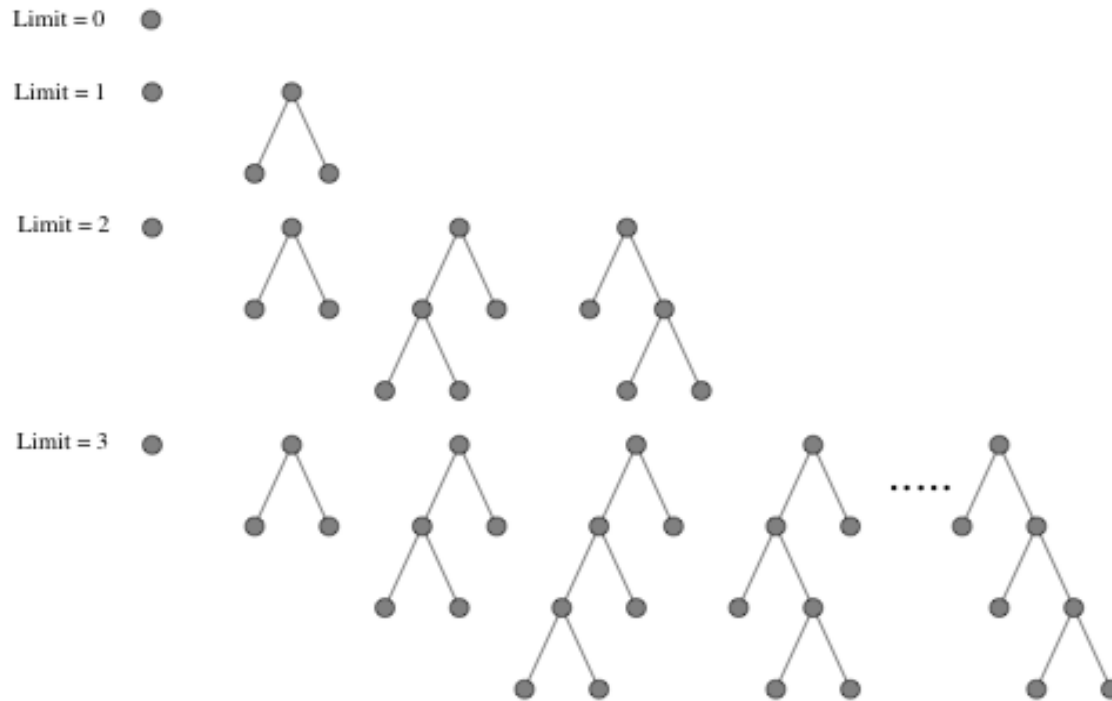
Always terminates:

- Avoids the problem of search never terminating by imposing a hard limit on the depth of any search path.

However, it is still **not complete** (the goal depth may be greater than the limit allowed.)

Iterative Deepening Search

Algorithm: Do depth-limited search, but with increasing depth.



Key Properties

$b \Rightarrow$ branching factor

$d \Rightarrow$ depth of shallowest solution

$m \Rightarrow$ maximum depth of the search tree

$\epsilon \Rightarrow$ minimum (positive) edge cost

$C^* \Rightarrow$ optimal path cost

	BFS	DFS	Uniform	IDS
Completeness (finite “b”)	Yes	No	Yes*	Yes
Optimality (step costs=1)	Yes	No	Yes	Yes
Optimality (general costs)	No	No	Yes	No
Space Complexity	$O(b^d)$	$O(bm)$	$O(b^{C^*/\epsilon})$	$O(bd)$
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^{C^*/\epsilon})$	$O(b^d)$

Generic Search Algorithm

Generic Search Algorithm

Initialize the search tree with the **initial state s_0** as root

Generic Search Algorithm

Initialize the search tree with the **initial state s_0** as root

Repeat:

 If no node can be expanded:

return Failure

Generic Search Algorithm

Initialize the search tree with the **initial state s_0** as root

Repeat:

 If no node can be expanded:

return Failure

 Choose a node for expansion, using some search strategy

Generic Search Algorithm

Initialize the search tree with the **initial state s_0** as root

Repeat:

If no node can be expanded:

return Failure

Choose a node for expansion, using some search strategy

If the node is the goal:

return Corresponding Path

Else:

For each applicable **action a** at the **state s** of the node:

Apply the transition **$(s, a) \rightarrow s'$** and add **s'** to the tree

Generic Search Algorithm

Initialize the search tree with the initial state s_0 as root

Repeat:

If no node can be expanded:

return Failure

Choose a node for expansion, using some search strategy

If the node is the goal:

return Corresponding Path

Else:

For each applicable action a at the state s of the node:

Apply the transition $(s, a) \rightarrow s'$ and add s' to the tree

Generic Search Algorithm

Initialize the search tree with the **initial state** s_0 as root

Repeat:

If no node can be expanded:

return Failure

Choose a node for expansion, using some search strategy

If the node is the goal:

return Corresponding Path

Else:

For each applicable **action** a at the **state** s of the node:

Apply the transition $(s, a) \rightarrow s'$ and add s' to the tree

What if we checked
the goal when
expanding?

This is called early vs
late goal evaluation.

Generic Search Algorithm

Initialize the search tree with the **initial state** s_0 as root

Repeat:

If no node can be expanded:

return Failure

Choose a node for expansion, using some search strategy

If the node is the goal:

return Corresponding Path

Else:

For each applicable **action** a at the **state** s of the node:

Apply the transition $(s, a) \rightarrow s'$ and add s' to the tree

How to deal with revisiting a state during expansion?

Quick Detour: Implementation Details

- Defining a search tree node:
 - A **state** from the planning problem
 - Additional information needed by the algorithm:
 - The **parent state pointer** and the action used to generate it
 - **Cost** of the path so far
 - Depth of the node
- Tree edges:
 - Correspond to the actions taken
 - Record of edges followed lives in node back pointers

Quick Detour: Implementation Details

- Search methods proceed by **expanding** a search tree node:
 - Start from a selected search tree node
 - Apply all legal operators to the state
 - Generating new nodes for all the corresponding successor states
- Critical property of expansion:
 - The nodes resulting from the action remain un-expanded until the algorithm later selects them. They must be stored somewhere.

Quick Detour: Implementation Details

- Need to keep track of the nodes to be expanded:
 - the **frontier of open nodes**.
- Implement this using a **queue**:

Quick Detour: Implementation Details

- Need to keep track of the nodes to be expanded:
 - the **frontier of open nodes**.
- Implement this using a **queue**:

Initialize the queue with the **initial state** s_0 node

Repeat:

 If queue is empty:

return Failure

 Dequeue a node

 If the node is the goal:

return Corresponding Path

 Else:

 For each applicable **action** a at the **state** s of the node:

 Apply the transition $(s, a) \rightarrow s'$ and add s' to the queue

Quick Detour: Implementation Details

- Need to keep track of the nodes to be expanded:
 - the **frontier of open nodes**.
- Implement this using a **queue**:

Initialize the queue with the **initial state** s_0 node

Repeat:

 If queue is empty:

return Failure

 Dequeue a node

 If the node is the goal:

return Corresponding Path

 Else:

 For each applicable **action** a at the **state** s of the node:

 Apply the transition $(s, a) \rightarrow s'$ and add s' to the queue

Search algorithms differ in their queuing function.

Assumptions (for next few lectures)

- **Static** (c.f. **dynamic**) environment
- **Observable** (c.f. **unobservable**) environment
- **Discrete** (c.f. **continuous**) states
- **Deterministic** (c.f. **stochastic**) environment

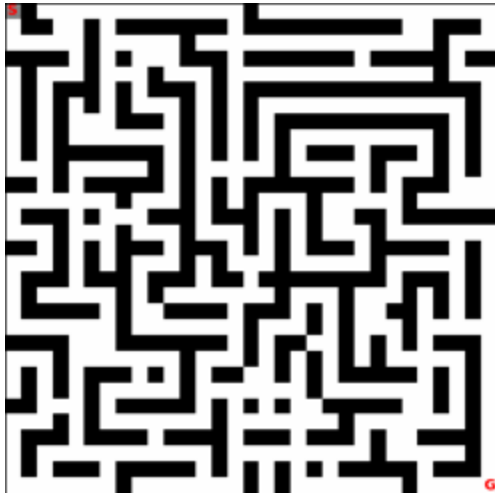
Today: simplified assumptions. Later, we will consider search in other settings.

Recap

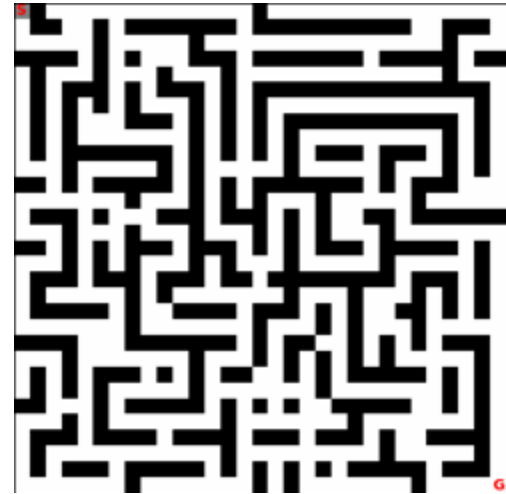
- Search in AI: Knowledge representation with state space models
- State space graphs and search trees
- A generic search algorithm: The ‘expansion’ operation
- Uninformed search:
 - BFS, DFS, Uniform, IDS

The Goal Today

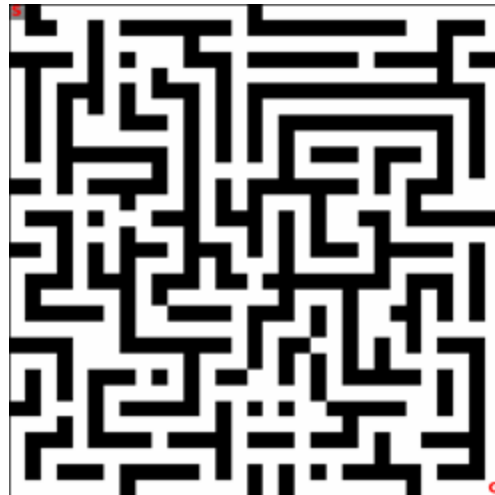
BFS



DFS

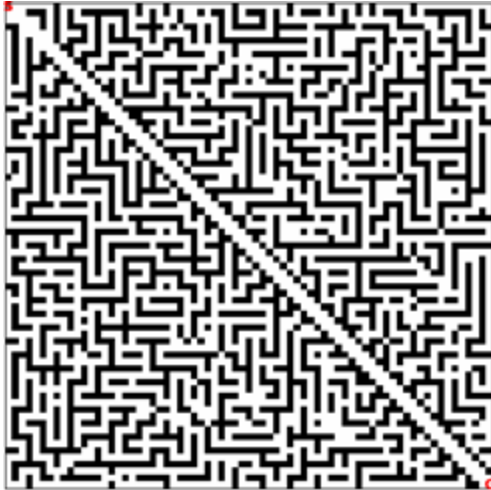


IDS

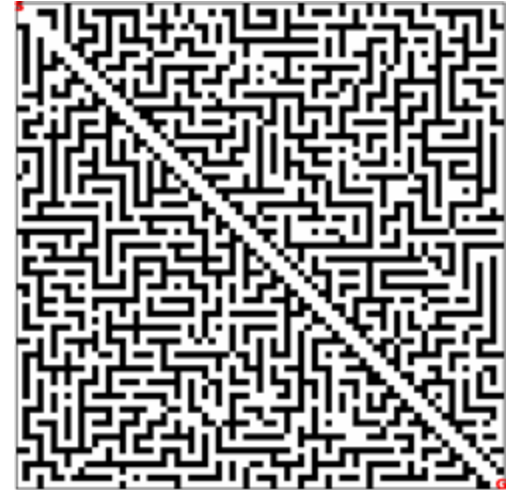


The Goal Today

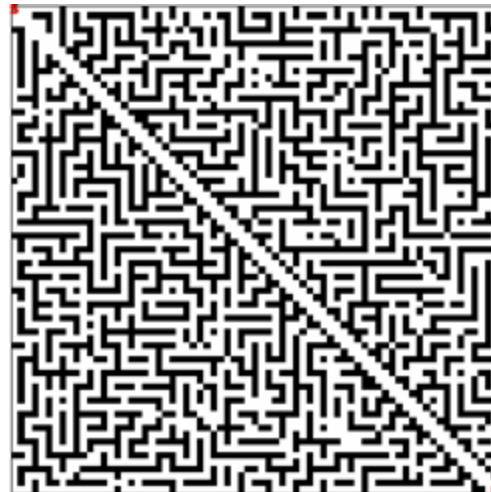
BFS



DFS



IDS



Next class

- Informed search
- **Starting next week (Sept 9), we'll be in MCMED 522!**